Lecture 3
Homographies, image stitching & two-view geometry

Robot Control · University of Warsaw

The content layout is greatly inspired by S. Nayar's First Principles of Computer Vision.

Three tools we already have

Homography (lecture 1)

$$\begin{bmatrix} \tilde{u}' \\ \tilde{v}' \\ \tilde{w}' \end{bmatrix} = \underbrace{\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}}_{H} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

the projective transformation: lines stay lines, parallelism is lost; defined up to scale → **8 DoF**

DLT (lecture 2)

Direct Linear Transform: fit a matrix defined up to scale (e.g. the camera matrix P) to correspondences $\tilde{\mathbf{u}} \equiv P\tilde{\mathbf{X}}$. Multiplying out the denominators gives equations **linear** in its entries $\mathbf{p}$; stacked:

$$A\mathbf{p} = \mathbf{0}: \quad \min \|A\mathbf{p}\| \text{ s.t. } \|\mathbf{p}\| = 1$$

→ eigenvector of the **smallest** eigenvalue of $A^\top A$

RANSAC (lecture 2)

wrong correspondences are normal: fit the model to **minimal** random samples, keep the one with the most inliers, refit

$$N \geq \frac{\log(1-p)}{\log(1-w^s)}$$

> Today all three come together, three times: for a homography, for the fundamental matrix, and for the pose between two frames.

Maps from a drone



the drone's latest photo

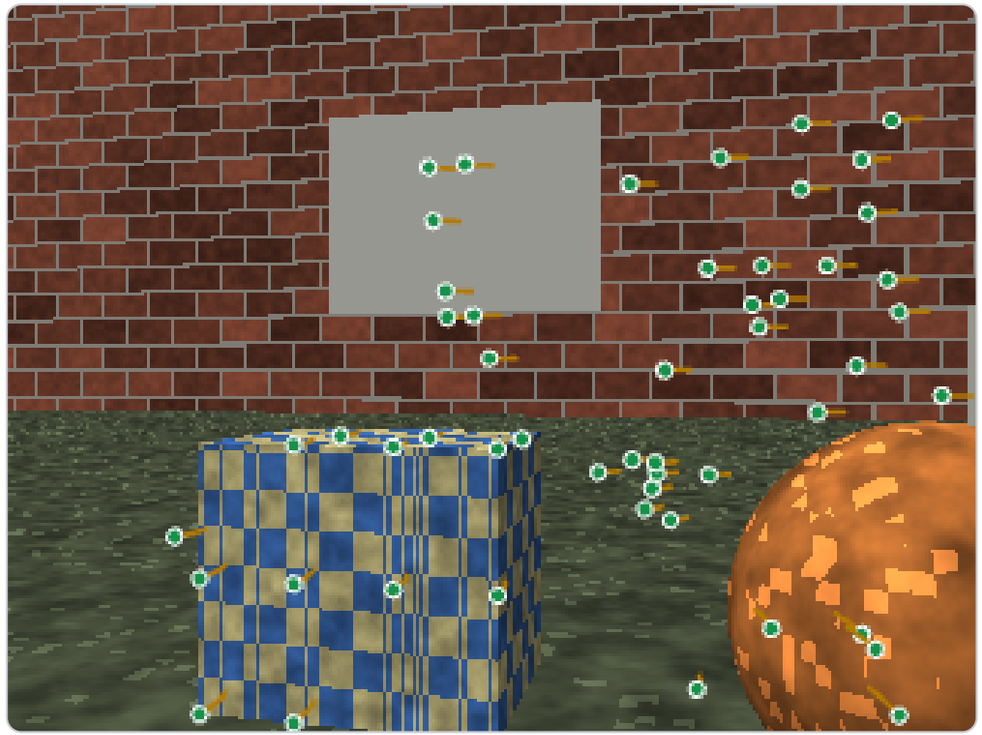


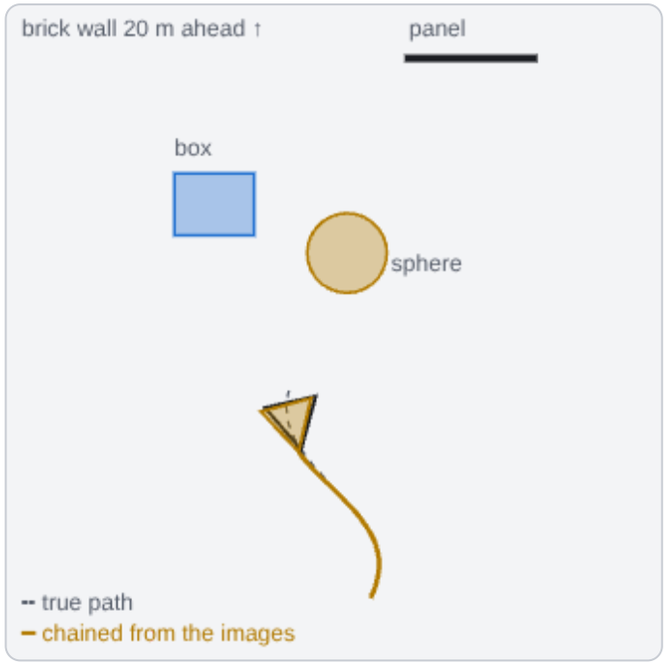
photo 32 of 54, flying west

A drone flies back and forth 150 m above a village and takes a photo every few seconds. Each photo is put onto the map by **one homography**, and the map grows as the drone flies. Part 1: why one homography is enough, how to estimate it, how to hide the seams.

orthophoto: DOP20, Bayerische Vermessungsverwaltung, geodaten.bayern.de (CC BY 4.0)

The camera as a motion sensor






```
frame 126 of 180, 52 tracked points
since the last frame: turned 0.25°,
  moved 1.6 cm
chained: 4 cm off after 2 m
```

Points tracked from frame to frame slide across the image: near points sweep long tracks, the far wall hardly moves. From these motions alone the camera's rotation and direction of travel can be computed (Part 3); chained over time they give its trajectory: visual odometry (Part 4). Small errors add up, so the chained path slowly drifts.

Today

1. **Image stitching**: when are two images related by a homography; estimating, warping, blending
2. **Features**: where the correspondences come from: detect, describe, match: classic and learned
3. **Two-view geometry**: a general camera pair; epipolar constraint, E, F, relative pose, the missing scale
4. **Visual odometry**: chaining two-view geometry over time, and why it drifts

For a robot

- maps from a drone (mosaics), a bird's-eye view of the floor
- recognising a place or an object from its features
- the camera as a motion sensor: how did I move between two frames?

Next lecture (Oct 28): 3D reconstruction (COLMAP, NeRF, Gaussian splatting) and SLAM.

Further reading: Szeliski, Computer Vision: Algorithms and Applications (2nd ed.), ch. 7 (features), 8 (alignment and stitching), 11 (two views, SfM); Hartley & Zisserman, Multiple View Geometry, ch. 4 (homography), 9 to 11 (epipolar geometry, F); First Principles of Computer Vision (S. Nayar).

Part 1

Image stitching

several photos → one wide image

Image stitching: the goal



Weighted average, weight = **distance to the image border** (a distance transform): smooth transitions.

weight of one image:



Geometry: warp the images so that corresponding points align: which transformation?

Photometry: blend them so that the borders are invisible (exposure, vignetting).

panorama: “Canary Wharf”, Andreas Mischok, Poly Haven (CC0)

Homography: definition

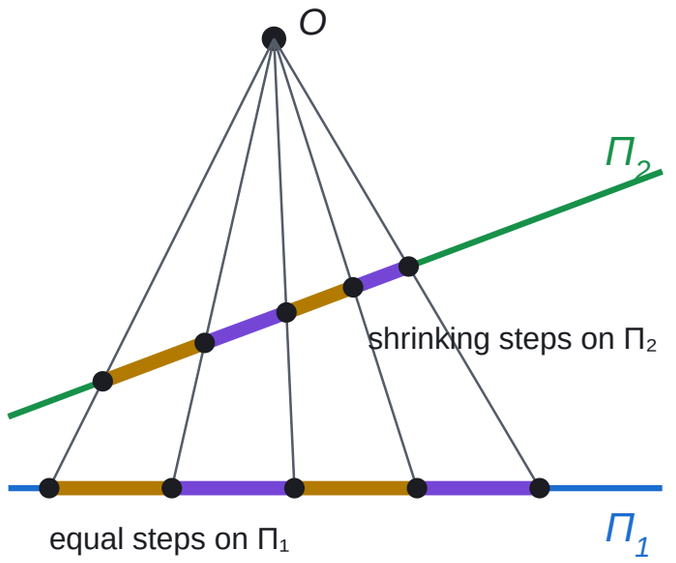
A **homography** (projective transformation) is an invertible 3×3 matrix H acting on homogeneous coordinates:

$$\begin{bmatrix} \tilde{u}' \\ \tilde{v}' \\ \tilde{w}' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \qquad \det H \neq 0$$

Back to pixels: divide by $\tilde{w}'$:

$$u' = \frac{\tilde{u}'}{\tilde{w}'} = \frac{h_{11}u + h_{12}v + h_{13}}{h_{31}u + h_{32}v + h_{33}}$$
$$v' = \frac{\tilde{v}'}{\tilde{w}'} = \frac{h_{21}u + h_{22}v + h_{23}}{h_{31}u + h_{32}v + h_{33}}$$

- H and kH ($k \neq 0$) give the same map: **8 DoF**
- straight lines stay straight; conversely, every invertible map of the (projective) plane that keeps lines straight is a homography (sketch: optional slide)
- closed under composition ($H_2 H_1$) and inverse (H^{-1}); $h_{31} = h_{32} = 0$: affine (lecture 1 hierarchy)



Geometric meaning: project one plane onto another through a point O (a central projection). Equal steps on Π_1 become shrinking steps on Π_2 (distances and ratios of lengths are not kept), but collinear points stay collinear. Why this is a homography: next (optional) slide.

Central projection is a homography: a sketch OPTIONAL

1. Plane → plane through O is a homography

Setting: points are vectors in $\mathbb{R}^3$, the centre $O = \mathbf{0}$; $\Pi_1, \Pi_2 \subset \mathbb{R}^3$ are planes not through O.

Coordinates on Π_1, chosen freely: a point $\mathbf{a}_1 \in \Pi_1$ (origin) and independent $\mathbf{b}_1, \mathbf{c}_1 \in \mathbb{R}^3$ parallel to Π_1 (axes). They define the map

$$\varphi_1 : \mathbb{R}^2 \to \Pi_1, \quad (s,t) \mapsto \mathbf{X} = \mathbf{a}_1 + s\,\mathbf{b}_1 + t\,\mathbf{c}_1 = \underbrace{[\,\mathbf{b}_1\ \mathbf{c}_1\ \mathbf{a}_1\,]}_{M_1 \in \mathbb{R}^{3\times 3}} \begin{bmatrix} s \\ t \\ 1 \end{bmatrix}$$

a bijection: every point $\mathbf{X} \in \Pi_1$ (a vector in $\mathbb{R}^3$) has exactly one pair $(s,t) \in \mathbb{R}^2$, its 2D coordinates (on an image plane: the pixel). M_1 is invertible: otherwise $\mathbf{a}_1 \in \operatorname{span}(\mathbf{b}_1, \mathbf{c}_1)$ and $O \in \Pi_1$. Likewise φ_2, M_2, (s',t') on Π_2.

The projection $\pi : \Pi_1 \to \Pi_2$ sends $\mathbf{X}$ to where the line $\{\lambda\mathbf{X} : \lambda \in \mathbb{R}\}$ through O meets Π_2: $\pi(\mathbf{X}) = \lambda\mathbf{X}$ for one number λ (depending on $\mathbf{X}$).

Claim: in coordinates, $(s,t) \mapsto (s',t') = \varphi_2^{-1}\big(\pi(\varphi_1(s,t))\big)$ is a homography:

$$M_2 \begin{bmatrix} s' \\ t' \\ 1 \end{bmatrix} = \lambda M_1 \begin{bmatrix} s \\ t \\ 1 \end{bmatrix} \Rightarrow \begin{bmatrix} s' \\ t' \\ 1 \end{bmatrix} \equiv \underbrace{M_2^{-1} M_1}_{H} \begin{bmatrix} s \\ t \\ 1 \end{bmatrix}$$

λ is the scale that homogeneous coordinates ignore. Other $\mathbf{a}, \mathbf{b}, \mathbf{c}$: H times invertible matrices, still a homography. Rays parallel to Π_2: $\tilde{w}' = 0$, the horizon.

2. Lines stay lines

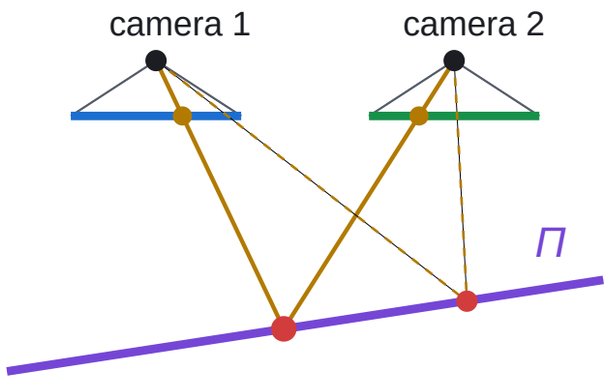
Geometry: a line of Π_1 and O span a plane, which meets Π_2 in a line. Algebra: if $\mathbf{l}^\top \tilde{\mathbf{u}} = 0$, then $(H^{-\top}\mathbf{l})^\top (H\tilde{\mathbf{u}}) = \mathbf{l}^\top \tilde{\mathbf{u}} = 0$: the image points lie on the line $H^{-\top}\mathbf{l}$.

3. Conversely: keeps lines ⇒ homography

- 4 points, no 3 collinear, and their 4 images determine a unique H (8 equations, 8 DoF: the DLT)
- let f be a bijection that keeps lines, and H agree with f on 4 such points; then $g = H^{-1} \circ f$ keeps lines and fixes the 4 points
- lines through fixed points and their intersections stay fixed; repeating this, g acts on each line like an automorphism of the field $\mathbb{R}$, and the only one is the identity: $g = \mathrm{id}$, so $f = H$

Hartley & Zisserman, sec. 2.3 (projective transformations, Theorem 2.10); 3 is the fundamental theorem of projective geometry.

When are two images related by a homography?

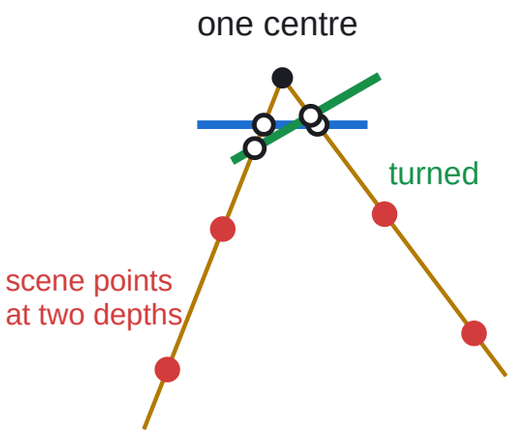


1. Planar scene

any camera motion

plane → image 1 is a homography H_1,

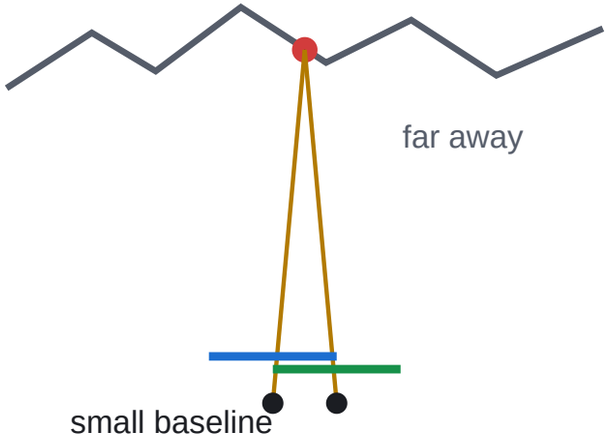
plane → image 2 is H_2, so image 1 → image 2 is $H_2 H_1^{-1}$



2. Pure rotation

any scene, the camera only turns about its centre

the rays do not change, only the image plane does: both red points on a ray land on one pixel (white), in either image



3. Distant scene

baseline ≪ depth

translation hardly matters: almost a pure rotation (aerial and landscape photos)

Otherwise (a 3D scene and a moving camera) there is **no** homography: points at different depths shift differently (parallax). That case is Part 3.

Why: the formulas

Pure rotation

Camera 1: $\tilde{\mathbf{u}}_1 \equiv K\mathbf{x}$. Camera 2 has the same centre, rotated: $\tilde{\mathbf{u}}_2 \equiv KR\,\mathbf{x}$.

$$\mathbf{x} \equiv K^{-1}\tilde{\mathbf{u}}_1 \;\Rightarrow\; \tilde{\mathbf{u}}_2 \equiv \underbrace{KRK^{-1}}_{H}\,\tilde{\mathbf{u}}_1$$

The depth of $\mathbf{x}$ cancels: one H for every pixel.

Planar scene

World frame on the plane, so $z_w = 0$ (lecture 2, the planar DLT degeneracy):

$$\tilde{\mathbf{u}} \equiv K\begin{bmatrix}\mathbf{r}_1 & \mathbf{r}_2 & \mathbf{r}_3 & \mathbf{t}\end{bmatrix}\begin{bmatrix}x_w\\ y_w\\ 0\\ 1\end{bmatrix} = \underbrace{K\begin{bmatrix}\mathbf{r}_1 & \mathbf{r}_2 & \mathbf{t}\end{bmatrix}}_{3\times 3\text{ homography}}\begin{bmatrix}x_w\\ y_w\\ 1\end{bmatrix}$$

Image 1 → image 2: $H_2 H_1^{-1}$. Zhang's calibration uses exactly these H_i.

General form (plane $\mathbf{n}^\top\mathbf{x}_1 = d$ in camera 1, camera 2 at $\mathbf{x}_2 = R\mathbf{x}_1 + \mathbf{t}$): $\mathbf{x}_2 = \left(R + \mathbf{t}\,\mathbf{n}^\top/d\right)\mathbf{x}_1$, so $H = K\left(R + \mathbf{t}\,\mathbf{n}^\top/d\right)K^{-1}$. For $d \to \infty$ (a distant plane) it becomes KRK^{-1}.

Distant scene: how far is far enough? OPTIONAL

Intuition

From a train window the posts by the track rush past, the trees behind them move slowly, and the mountains on the horizon hardly move (the Moon seems to travel with the train). When the camera **moves**, the image shift shrinks with the depth; when it **turns**, everything shifts alike.

Formula

A point at depth Z in camera 1: $\mathbf{x}_1 = Z\,K^{-1}\tilde{\mathbf{u}}_1$; in camera 2: $\mathbf{x}_2 = R\mathbf{x}_1 + \mathbf{t}$. So

$$\tilde{\mathbf{u}}_2 \equiv K\mathbf{x}_2 = Z\,KRK^{-1}\tilde{\mathbf{u}}_1 + K\mathbf{t} \;\equiv\; \underbrace{KRK^{-1}}_{\text{rotation } H}\,\tilde{\mathbf{u}}_1 + \frac{1}{Z}\,K\mathbf{t}$$

The translation enters only through $\mathbf{t}/Z$. Using $H = KRK^{-1}$ anyway leaves an error (parallax) of about $f\,b/Z$ pixels for a sideways baseline b: the disparity of lecture 2.

Rule of thumb

Scene from $Z_{\min}$ to ∞: one H is good when $f\,b/Z_{\min} \lesssim 1$ px (or the matching tolerance). A plane at depth d instead, $H = K(R + \mathbf{t}\mathbf{n}^\top/d)K^{-1}$, leaves $f\,b\,|1/Z - 1/d|$.

Examples

- hand-held panorama with a phone: $f \approx 3000$ px, the lens moves $b \approx 2$ cm between shots: a building 60 m away 1 px, 20 m away 3 px, a person 2 m away **30 px** (ghosting)
- drone at $A = 100$ m, $f = 3650$ px, photos $b = 20$ m apart: the ground and the top of a 10 m tree differ by $3650 \cdot 20 \cdot (\frac{1}{90} - \frac{1}{100}) \approx 80$ px: not distant; the ground is a **plane** (case 1), trees and roofs break it (later)

Pure rotation: the homography is exact

image 1 (reference camera)

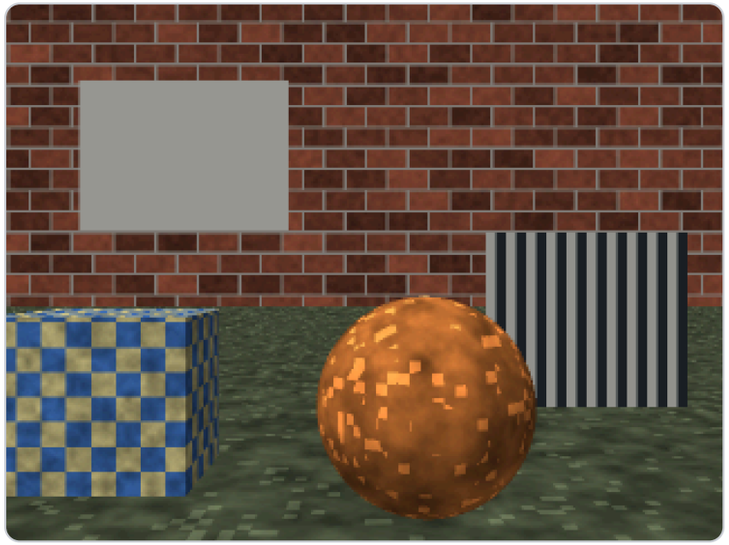


image 2 (moved camera)

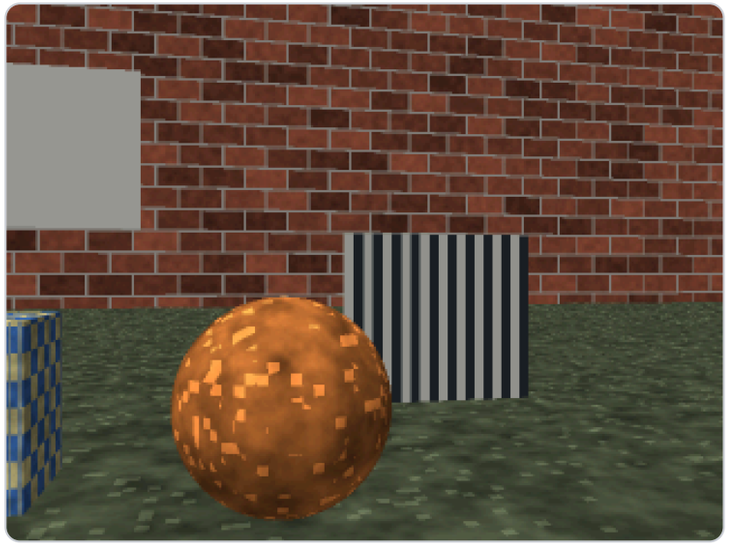
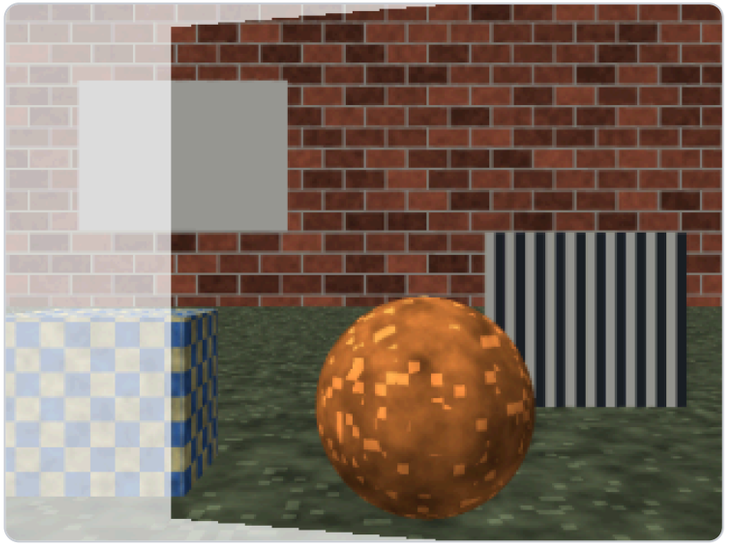


image 1 + warped image 2 (50/50)



pure rotation by 12°: $H = K R K^{-1}$ holds for every pixel, whatever its depth

Translation: parallax

image 1 (reference camera)

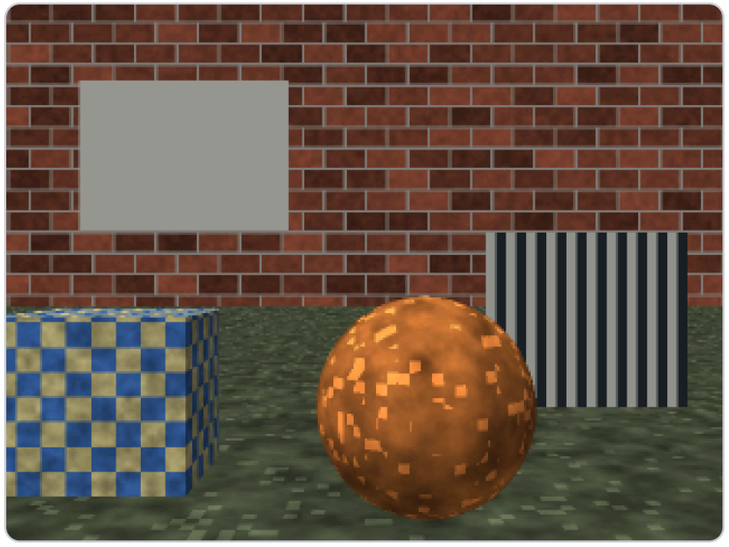


image 2 (moved camera)

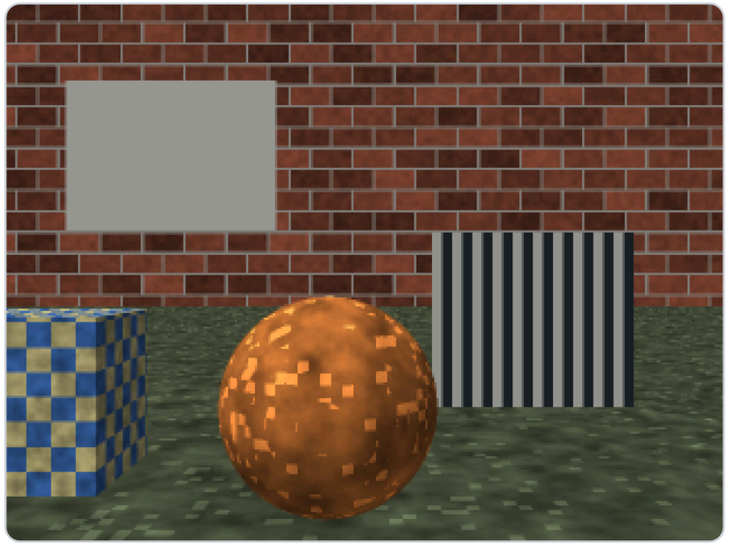
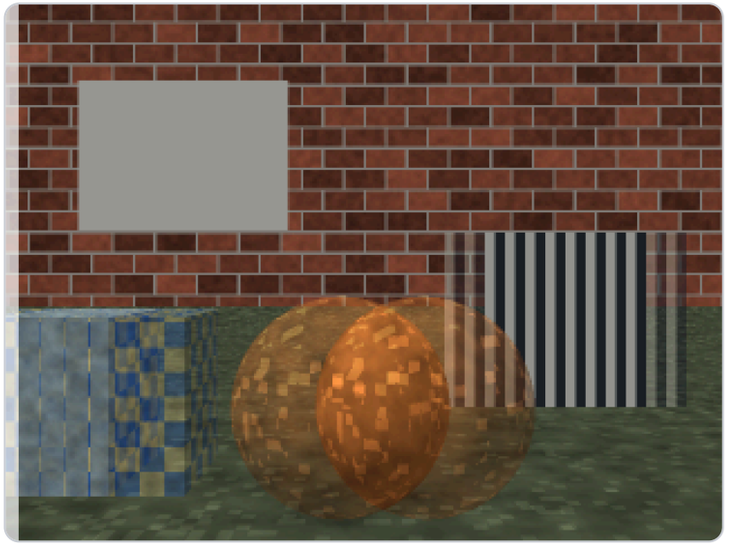


image 1 + warped image 2 (50/50)



translation by 0.4 m: the back-wall plane's H aligns only that plane, everything else appears twice

wall H: the homography of the back wall, the plane $z = 20$ m in camera 1 ($\mathbf{n} = (0, 0, 1)$, $d = 20$): $H = K\left(R + \mathbf{t}\,\mathbf{n}^\top / d\right) K^{-1}$. Exact for the wall; nearer points shift more than it predicts and appear twice.

ground H: the same formula for the ground, the plane $y = 1$ m below the camera ($\mathbf{n} = (0, 1, 0)$, $d = 1$; y points down). Now the ground aligns and the wall, far from that plane, is doubled.

With translation every plane has **its own** homography: no single H fits a 3D scene. With pure rotation $\mathbf{t} = \mathbf{0}$, the plane drops out and one $H = KRK^{-1}$ fits everything.

Estimating a homography: H is the unknown

So far H was given and we computed where a point goes. Now the **points are known** (correspondences $(u_i, v_i) \to (u_i', v_i')$) and the **9 entries of H are the unknowns**. As with the projection matrix P in lecture 2, flatten H into one vector:

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \longrightarrow \mathbf{h} = \begin{bmatrix} h_{11} & h_{12} & h_{13} & h_{21} & h_{22} & h_{23} & h_{31} & h_{32} & h_{33} \end{bmatrix}^\top$$

One correspondence, the u' equation; multiply out the denominator and collect the terms:

$$u' = \frac{h_{11}u + h_{12}v + h_{13}}{h_{31}u + h_{32}v + h_{33}}$$

$$0 = u\,h_{11} + v\,h_{12} + 1\,h_{13} - u'u\,h_{31} - u'v\,h_{32} - u'\,h_{33}$$

The numbers u, v, u' are now **coefficients**: one linear equation in $\mathbf{h}$. With the v' equation, two rows per correspondence:

$$\begin{bmatrix} u & v & 1 & 0 & 0 & 0 & -u'u & -u'v & -u' \\ 0 & 0 & 0 & u & v & 1 & -v'u & -v'v & -v' \end{bmatrix} \mathbf{h} = \mathbf{0}$$

The roles swap

	known	unknown
warping	H, (u, v)	(u', v')
estimating	(u_i, v_i), (u_i', v_i')	$\mathbf{h}$ (9 numbers)

Why it is linear

u' is a **ratio** of two expressions in H; cross-multiplying removes the ratio. The price: the equations are homogeneous, so $\mathbf{h}$ is found only up to scale, exactly as H is defined.

Estimating a homography: solving for $\mathbf{h}$

Stack the two rows of every correspondence:

$$\underbrace{\begin{bmatrix} u_1 & v_1 & 1 & 0 & 0 & 0 & -u'_1 u_1 & -u'_1 v_1 & -u'_1 \\ 0 & 0 & 0 & u_1 & v_1 & 1 & -v'_1 u_1 & -v'_1 v_1 & -v'_1 \\ & & & & \vdots & & & & \\ u_n & v_n & 1 & 0 & 0 & 0 & -u'_n u_n & -u'_n v_n & -u'_n \\ 0 & 0 & 0 & u_n & v_n & 1 & -v'_n u_n & -v'_n v_n & -v'_n \end{bmatrix}}_{A:\, 2n \times 9} \mathbf{h} = \mathbf{0}$$

- $\mathbf{h} = \mathbf{0}$ always solves it, and $\mathbf{h}$ is defined up to scale: $\min \|A\mathbf{h}\|$ s.t. $\|\mathbf{h}\| = 1$ (lecture 2)
- solution: the right singular vector of A for the smallest singular value
- reshape $\mathbf{h}$ back into H: rows of three

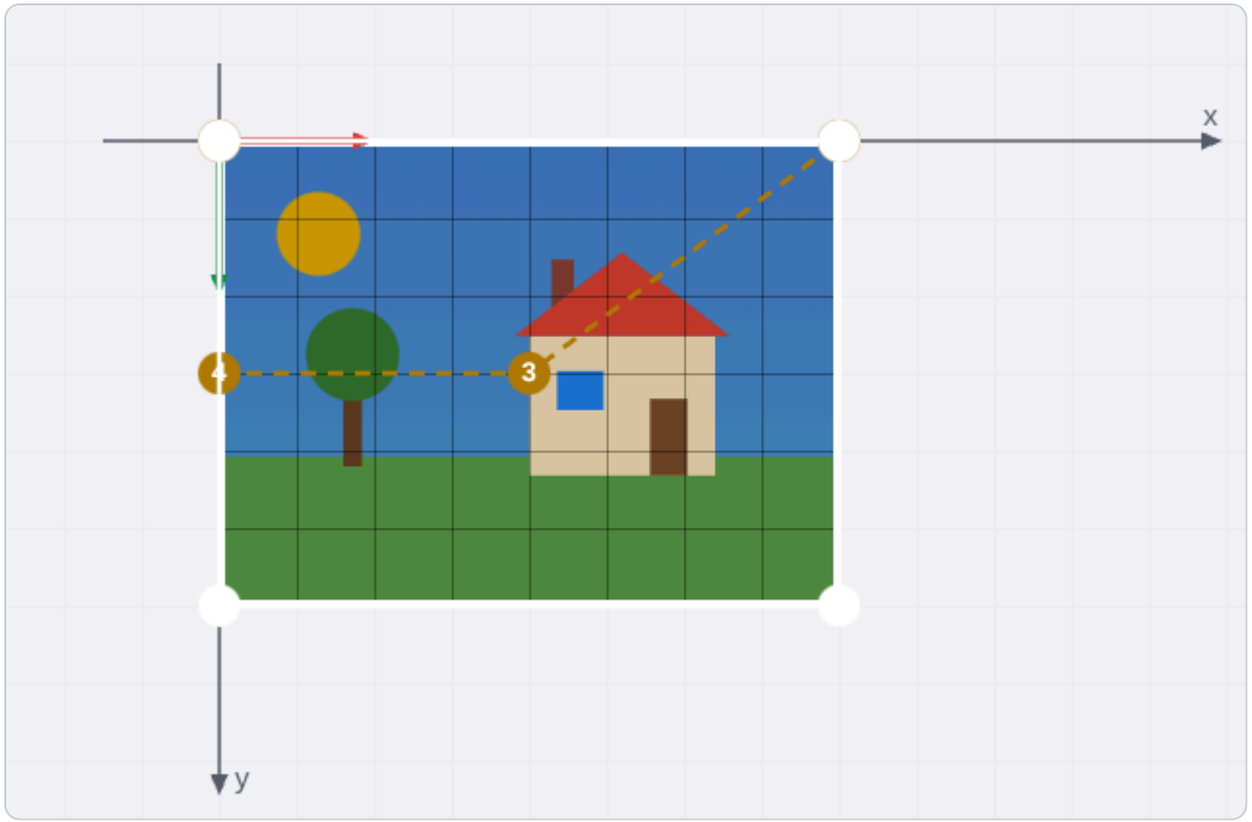
How many points?

8 DoF, 2 equations per point → **4 points**, no three of them collinear.

In practice

- **normalise** first: shift the points to their centroid, scale to mean distance $\sqrt{2}$ (Hartley): pixel values $\sim 10^3$ next to 1 make A badly conditioned
- refine by minimising the reprojection error in pixels
- with wrong matches: RANSAC with $s = 4$
- `cv2.findHomography(src, dst, cv2.RANSAC, 3.0)`

Exercise: a projective warp by hand



Task 1 | Task 2 | Task 3

Change only the third row: keep the top edge, send the bottom corners to the marked points (a floor seen in perspective).

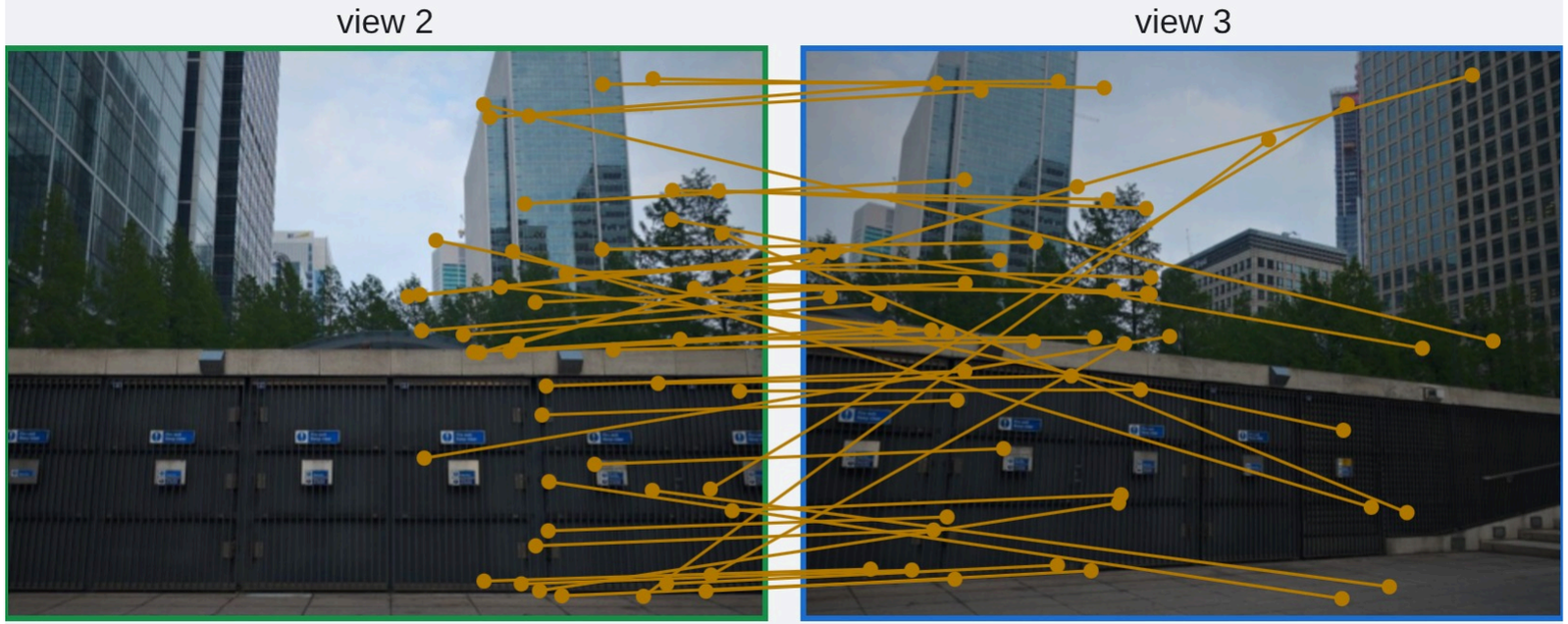
$$M = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

✗ not yet

`max corner error: 200 px`

`rigid (rotation + translation) — origin fixed (linear)`

Matches and RANSAC



50 matches between view 2 and view 3.

From a feature matcher: most are right, but 15 (30%) are wrong. Which ones?

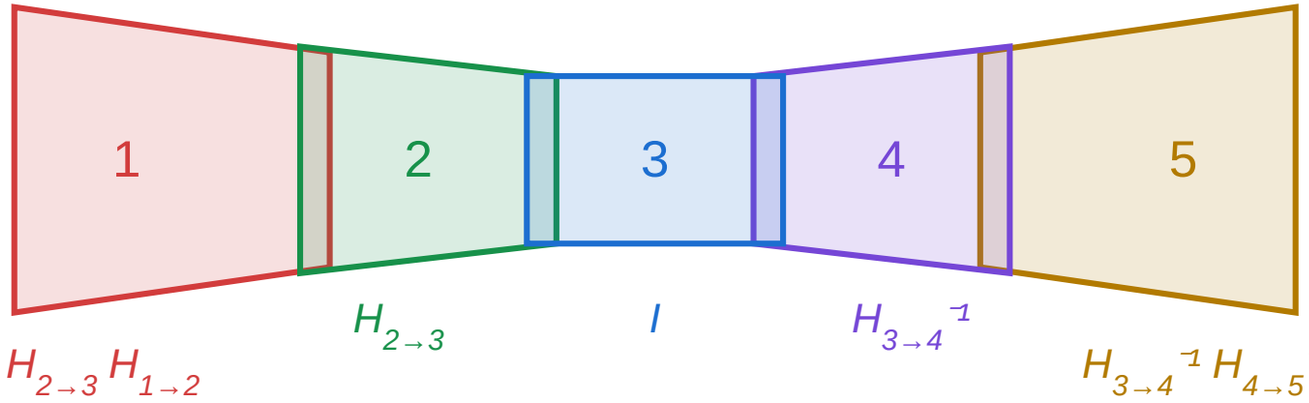
panorama: “Canary Wharf”, Andreas Mischok, Poly Haven (CC0)

Composing the homographies

photos; neighbours overlap, so only their homographies are estimated (matches + RANSAC):



the panorama in the frame of the reference photo 3: compose along the chain (read right to left), invert to go backwards:



outer photos:
more stretched

Homographies compose by matrix products: $H_{1\to3} = H_{2\to3}\,H_{1\to2}$ (first $1 \to 2$, then $2 \to 3$), and $H_{4\to3} = H_{3\to4}^{-1}$. The canvas adds a translation T so that all coordinates are positive: photo i is warped by $T\,H_{i\to3}$.

Errors multiply along the chain: the outer photos are the least accurate. Choosing the **middle** photo as the reference keeps the chains short and the stretching symmetric.

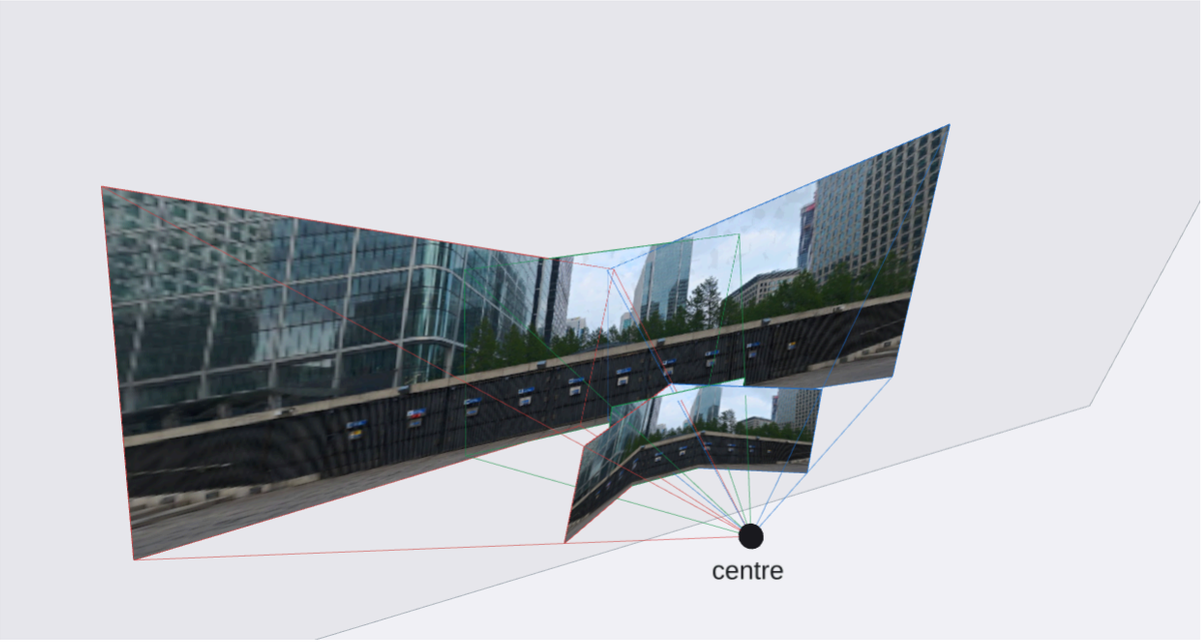
Warping into a common frame

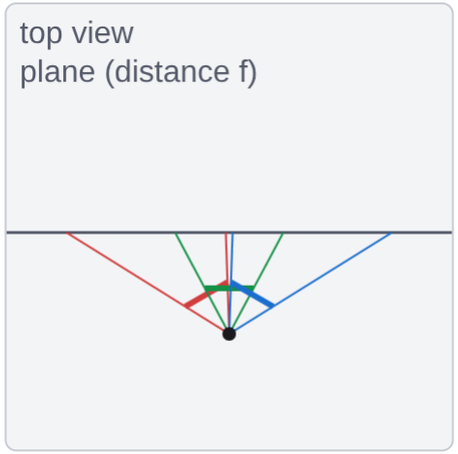
- pick a **reference** image (the middle one): the others are warped into its frame
- compose the pairwise homographies into $H_{i\to\text{ref}}$ (previous slide)
- size of the result (the canvas): warp the four corners of every image; their bounding box $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ is in the reference frame, and $x_{\min}, y_{\min} < 0$ (photos left of / above the reference). Canvas pixels start at $(0, 0)$, so shift by $T = \begin{bmatrix} 1 & 0 & -x_{\min} \\ 0 & 1 & -y_{\min} \\ 0 & 0 & 1 \end{bmatrix}$: photo i is warped by $TH_{i\to\text{ref}}$
- **backward** warping (lecture 1): for every output pixel, H^{-1} gives the source position; interpolate (bilinear)

Many images

- chaining accumulates errors: the last image drifts (the same problem as odometry, Part 4)
- remedy: **bundle adjustment**: adjust all rotations (and f) together so that all matches agree (Brown & Lowe, AutoStitch, 2007)
- wide panoramas (> ~120°) stretch without bound on one plane: project onto a **cylinder** or a **sphere** instead

Warping onto the reference plane, in 3D





edges at ±58.1°;
panorama width:
plane 2f·tan: 1444 px
cylinder 2f·angle:
912 px
f = 450 px; one
image: 480 px

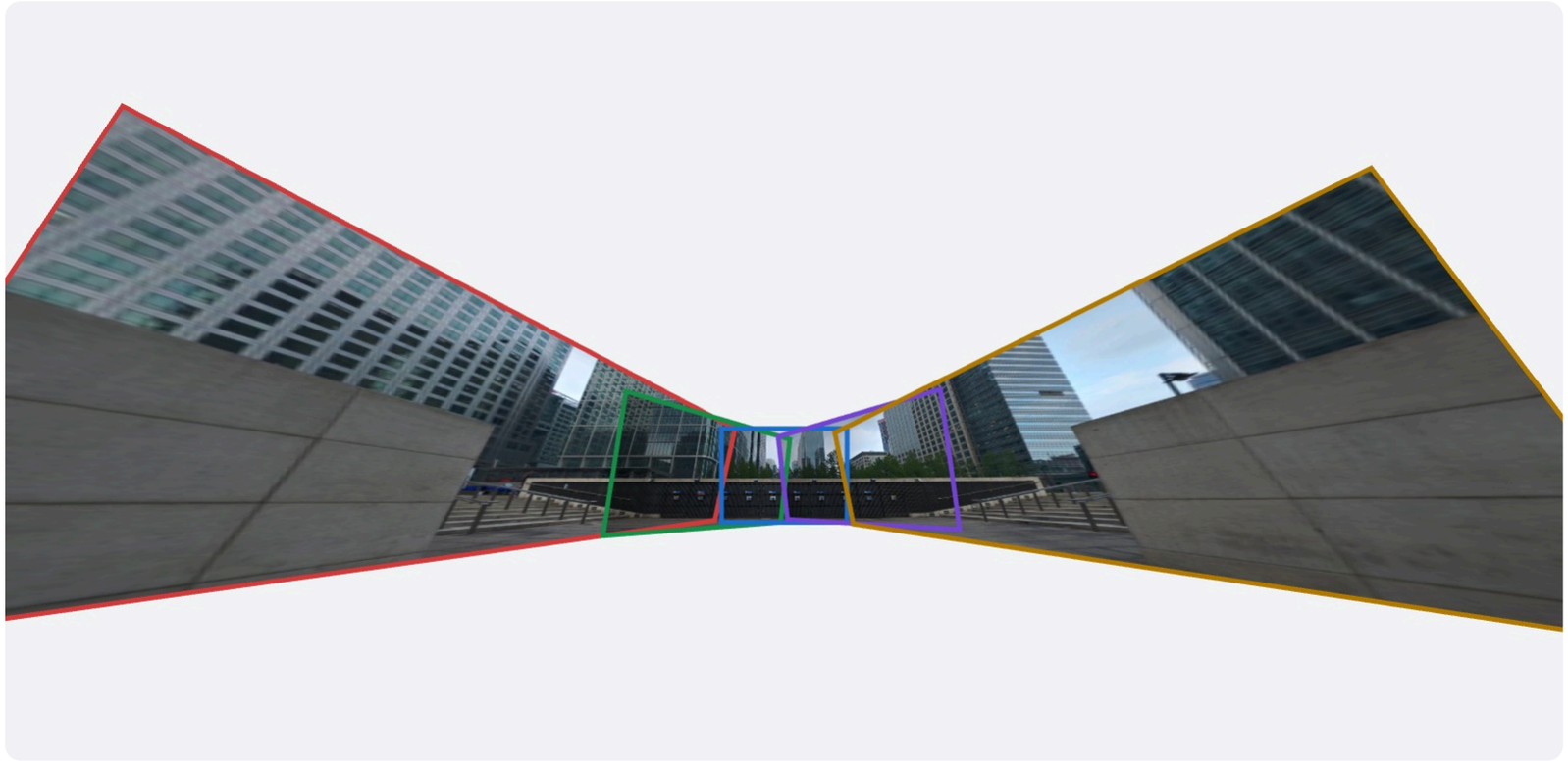
One camera centre, three photos (small frames) taken by turning it. Every pixel of photo i defines a ray; the panorama pixel is where the ray meets the **image plane of the reference photo**, extended.

This is exactly the homography $H = K R_{\text{ref}} R_i^{\top} K^{-1}$: the same rays, a different plane.

Turn the outer photos further: their projections stretch, and at 90° their rays run parallel to the plane and never meet it.

panorama: “Canary Wharf”, Andreas Mischok, Poly Haven (CC0)

Wide panoramas: plane or cylinder



Every view is **backward-warped** onto the image plane of view 3, with H chained between neighbours.

Later views overwrite earlier ones. Coloured: the warped borders.

mean error vs the true H (in source pixels):

$H_{1\to3}$: 2.13 px

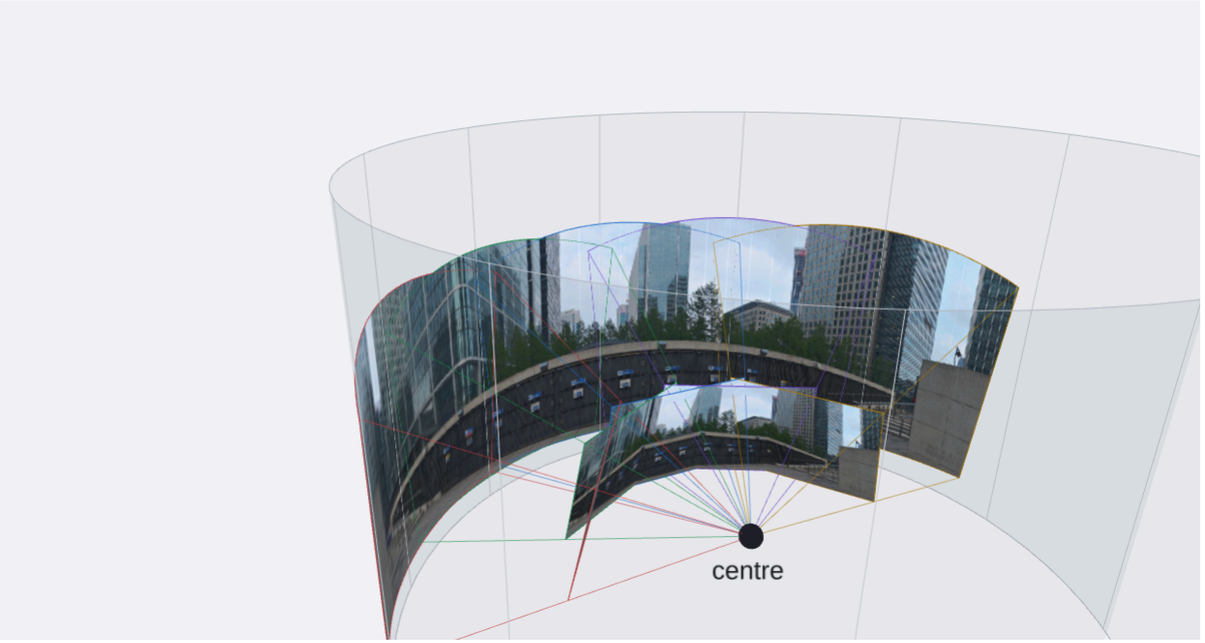
$H_{2\to3}$: 0.38 px

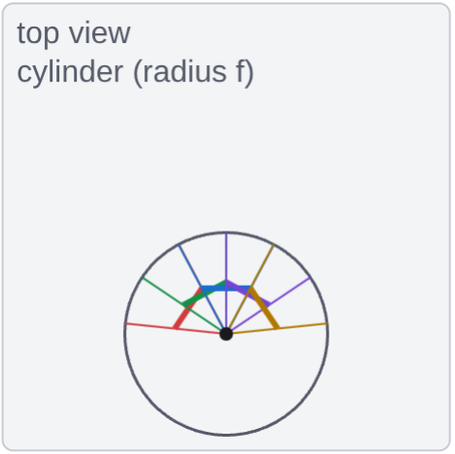
$H_{4\to3}$: 1.99 px

$H_{5\to3}$: 6.84 px

panorama: “Canary Wharf”, Andreas Mischok, Poly Haven (CC0)

Plane or cylinder, in 3D OPTIONAL





edges at ±84.1°;
panorama width:
plane 2f·tan: 8668 px
cylinder 2f·angle:
1321 px
f = 450 px; one
image: 480 px

Five photos, turned up to ±56°. On the **plane** the outer ones explode in size; on a **cylinder** around the camera every direction is equally far away, so all photos keep their size.

"unroll" flattens the cylinder into the panorama strip, $u = f\,\theta$: a turn of the camera becomes a horizontal shift.

The price: the strip is no longer a pinhole image, so straight horizontal lines bend (next slide).

panorama: “Canary Wharf”, Andreas Mischok, Poly Haven (CC0)

Panorama surfaces: the model OPTIONAL

Backward warping: panorama pixel (u, v) → a **ray** $\mathbf{d}$ in the reference camera frame → its pixel in view i: $\tilde{\mathbf{u}}_i \equiv K R_i \mathbf{d}$.
The surfaces differ only in pixel → ray:

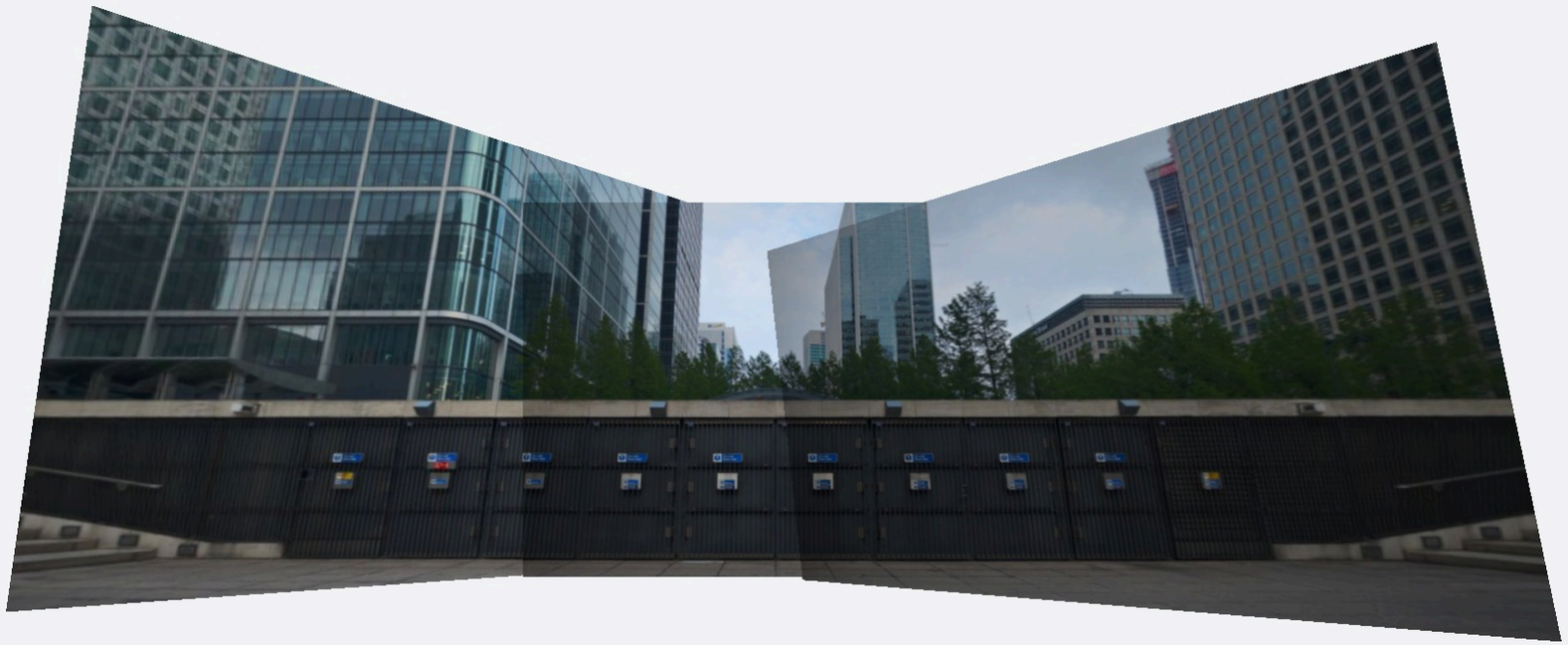
	plane (reference image plane)	cylinder (radius f, vertical axis)	sphere (equirectangular)
coordinates	u, v on the plane $z = f$	angle $\theta = (u - o_x)/f$, height $h = (v - o_y)/f$	$\theta = (u - o_x)/f$, $\varphi = (v - o_y)/f$
ray $\mathbf{d}$	$(u - o_x,\ v - o_y,\ f) \equiv K^{-1}\tilde{\mathbf{u}}$	$(\sin\theta,\ h,\ \cos\theta)$	$(\cos\varphi \sin\theta,\ \sin\varphi,\ \cos\varphi \cos\theta)$
view i	$K R_i K^{-1}\tilde{\mathbf{u}}$: a **homography**	not a homography; a turn about the vertical axis is a **horizontal shift** $\Delta u = f\,\Delta\theta$	
straight lines	all stay straight (it is a pinhole image)	vertical ones stay straight; horizontal ones bend (except the horizon)	only the vertical ones and the horizon
field of view	$< 180^\circ$; the stretch at angle α off-axis is $1/\cos^2\alpha$: ×4 at 60°, ×33 at 80°	360° horizontally; vertically as a plane	360° × 180°; the poles are stretched
needs	only the H's (no calibration)	f (and K), and the rotation axis = the cylinder axis (a level camera)	

Plane makes sense for a few photos (under ~90°) and is the **only** correct choice for a planar scene (a floor, a facade, a document): there the images are related by H, not by a rotation.

Cylinder: a horizontal sweep from one spot (phone panoramas, a rotating robot head). Wrong for a planar scene photographed while walking.

Sphere: full 360° cameras, VR; the format of the photo the panorama on these slides is rendered from.

Warping and blending to play with



Each pixel from one image (the last one): **hard seams** from exposure and vignetting.

panorama: “Canary Wharf”, Andreas Mischok, Poly Haven (CC0)

Blending

The images differ in exposure, vignetting and white balance; small misalignments remain. Overwriting gives **visible seams**.

Feathering: a weighted average, weights falling off towards each image's border:

$$I(\mathbf{x}) = \frac{\sum_i w_i(\mathbf{x})\, I_i(\mathbf{x})}{\sum_i w_i(\mathbf{x})}$$

$w_i(\mathbf{x})$: distance from $\mathbf{x}$ to the border of image i: a **distance transform** of the image's mask (`cv2.distanceTransform`).

Better blending

- **exposure compensation**: one gain per image, fitted on the overlaps
- **seam finding**: cut where the images agree (graph cut): hides parallax and moving people
- **multi-band blending** (Burt & Adelson, 1983): blend low frequencies over a wide band, details over a narrow one: no ghosting, no seams

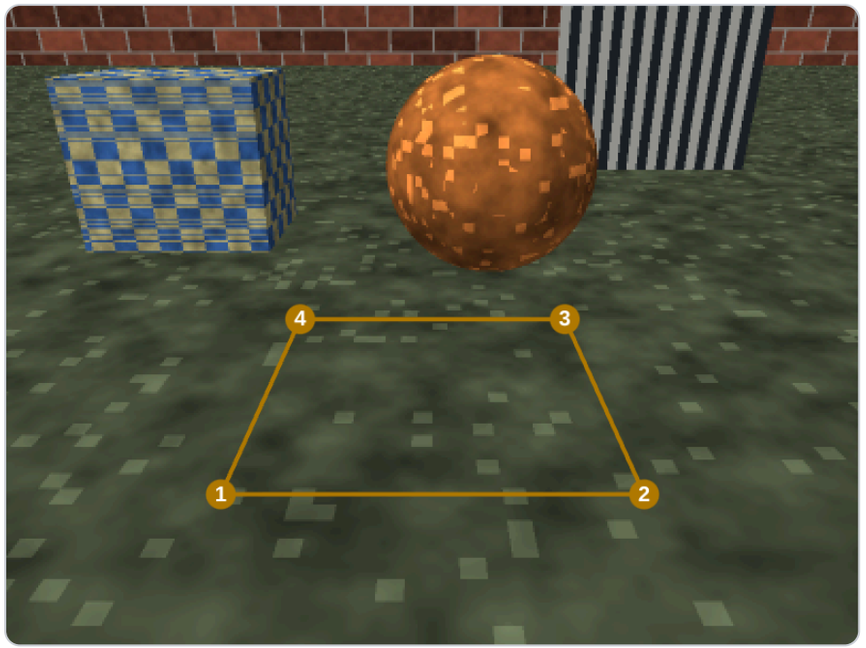
All of it: `cv2.Stitcher_create()`.

Homographies in robotics

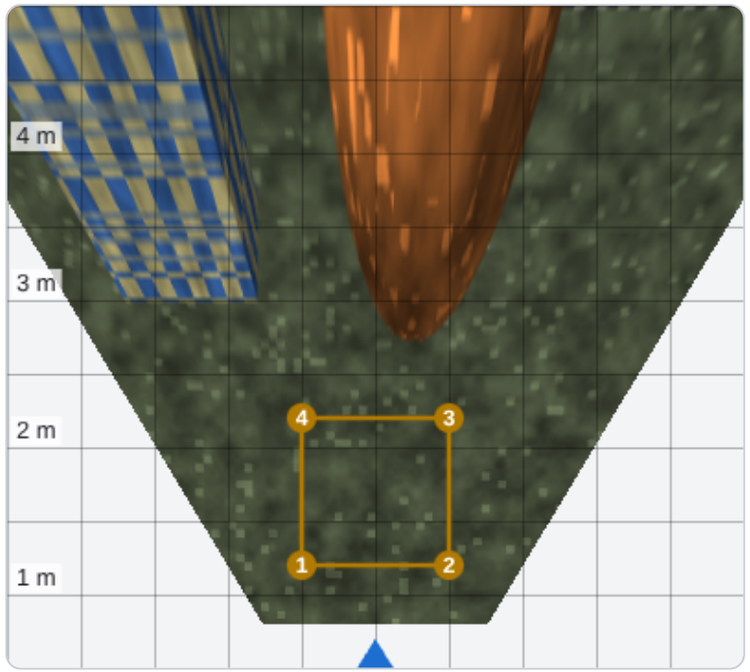
- **bird's-eye view** of the floor or the road (inverse perspective mapping): the ground is a plane, so camera → top view is one homography, from 4 marked floor points; parking assistants, lane detection, mobile robots
- **aerial mosaics**: a drone far above the ground (cases 1 and 3), maps for agriculture and inspection. For flat ground this is **orthorectification** (orthophotos, orthomosaics); hilly terrain needs a terrain model instead of one homography
- **planar markers**: the 4 corners of an ArUco/AprilTag give a homography; the pose follows from $H = K[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}]$ (lecture 2, IPPE)
- **calibration**: Zhang's method, one homography per view of the board (lecture 2)
- **stereo rectification**: one homography per image makes the epipolar lines horizontal (lecture 2 → Part 3)
- **image stabilisation**: undo the camera's small rotations between frames with KRK^{-1} (the rotation from a gyroscope)

Bird's-eye view: one homography of the floor

camera image (robot camera, 1 m above the floor)



top view: one homography, 1 grid cell = 0.5 m




```
H (ground → image)
   710     429     200
     0    21.7     793
     0   2.145       1
```

From the geometry: H = K [r_1 r_3 r_2], camera 1 m above the ground, pitch 25°.

4-point DLT from the marked square gives the same H (max difference < 0.01 px).

The floor is a plane, so camera image → top view is a single homography: here from the camera's height and pitch, or from 4 marked floor points. The floor comes out metric (the marked square is a square, distances can be read off the grid). Everything above the floor (box, sphere) is smeared away from the camera: the homography holds only for points on that plane.

Aerial mosaics: photogrammetry flights

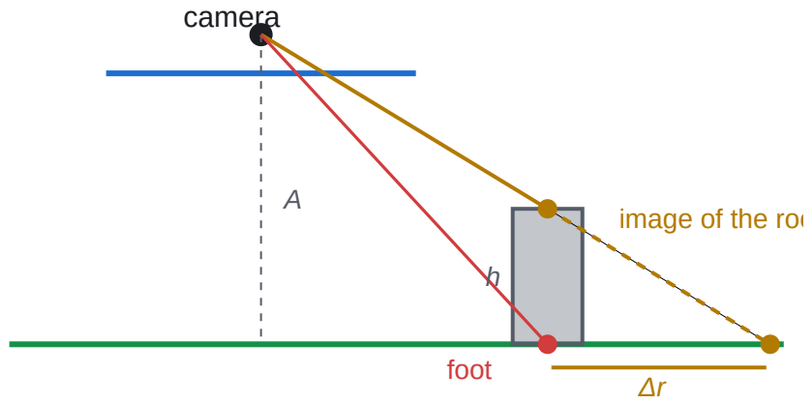
The flight

- a drone flies parallel lines back and forth (a "lawnmower" pattern), camera pointing straight down (nadir)
- overlap: about 75 to 80% along a line, 60 to 70% between lines: every ground point is in several photos
- **ground sampling distance** (the size of one pixel on the ground): $\mathrm{GSD} = A/f$, with A the altitude and f in pixels. Example: $f \approx 3650$ px (1-inch sensor, 8.8 mm lens), $A = 100$ m $\rightarrow$ 2.7 cm/px

Flat ground: homographies

- photo $i \leftrightarrow$ ground plane: $H_i = K[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}]$ (case 1), the pose from GPS + IMU
- refined with feature matches between overlapping photos and bundle adjustment; ground control points or RTK GPS give centimetre accuracy

Real terrain: relief displacement



- a point at height h maps through the ground homography as if it were on the ground, shifted away from the image centre by $\Delta r = r\,h/A$ (r: its distance from the centre in the image)
- buildings and trees lean outwards, differently in each photo: the mosaic breaks there
- the real pipeline (next lecture): structure from motion $\rightarrow$ 3D points $\rightarrow$ surface model $\rightarrow$ a **true orthophoto**, every point seen straight from above; OpenDroneMap (open source), Pix4D, Metashape

Aerial mosaic, as the drone flies



the drone's latest photo



photo 32 of 54, flying west

altitude 150 m, nadir camera
footprint 136 × 102 m
overlap 70% along, 60% across
6 lines × 9 photos

H (map m → photo px), h_{33} = 1

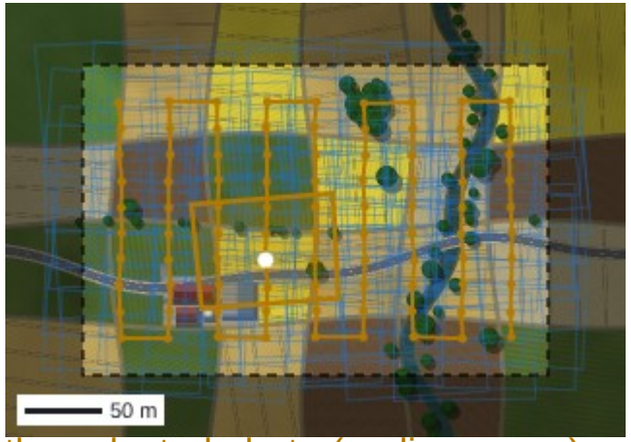
-2.84	-0.1726	938
0.1347	-2.88	637
0	-0.0002	1

Every new photo: its pose gives $H_i = K[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}]$ (readout), each map pixel is sent through H_i into the photo (backward warping), and the photos are feathered. Westbound photos are turned by 180°: H_i contains that rotation. Exact poses here; real GPS/IMU poses: next slide.

orthophoto: DOP20, Bayerische Vermessungsverwaltung, geodaten.bayern.de (CC BY 4.0)

Aerial mosaic to play with

flight plan: footprints of all photos



the selected photo (nadir camera)

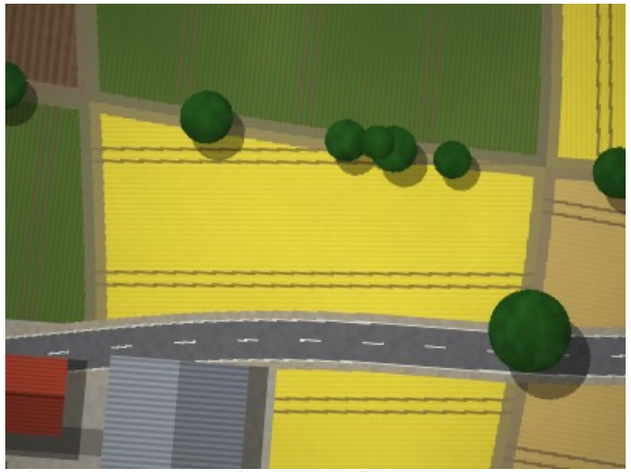


photo 34 of 90 (line 4)

mosaic: each photo warped to the ground by its homography



altitude **100 m**
footprint 90 × 68 m
GSD = A / f = **22.5 cm/px**
overlap: front 75%, side 65%
photos every 16.9 m, lines 31.5 m apart
90 photos (9 lines × 10)
mosaic: ground homography of each photo, $H_i = K\,[r_1\ r_2\ t]$, from exact GPS + IMU poses

Each photo goes onto the map by one homography $H_i = K[\mathbf{r}_1\ \mathbf{r}_2\ \mathbf{t}]$ from the drone's GPS and IMU pose. Try "GPS/IMU noise" (seams: hence feature matching and bundle adjustment) and "buildings / trees (3D)" (tall objects move away from each photo's centre by $\Delta r = r\,h/A$).

Part 2

Features

where do the correspondences come from?

Detect, describe, match

detect distinctive points → **describe** a vector per point → **match** nearest descriptors → **verify** RANSAC on the geometry

photo 1

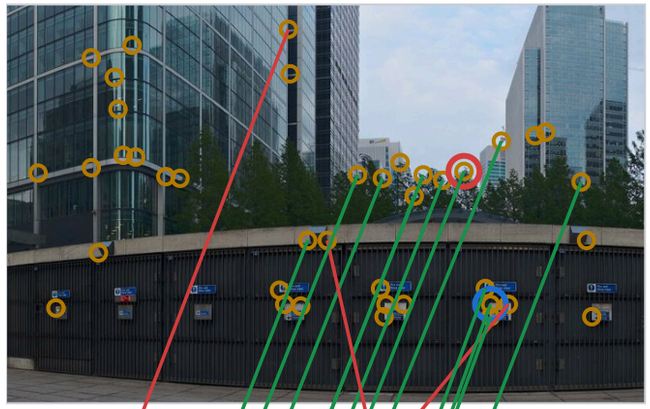
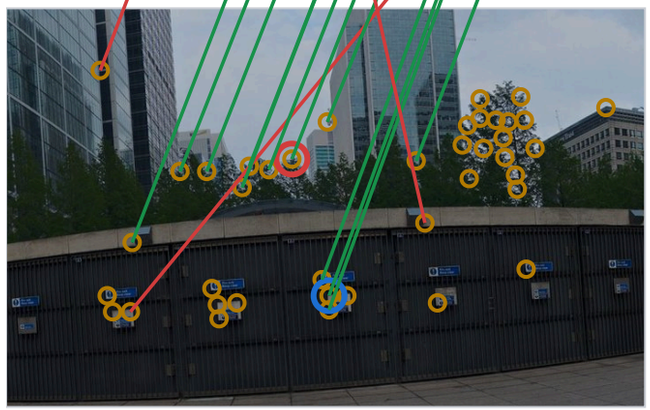
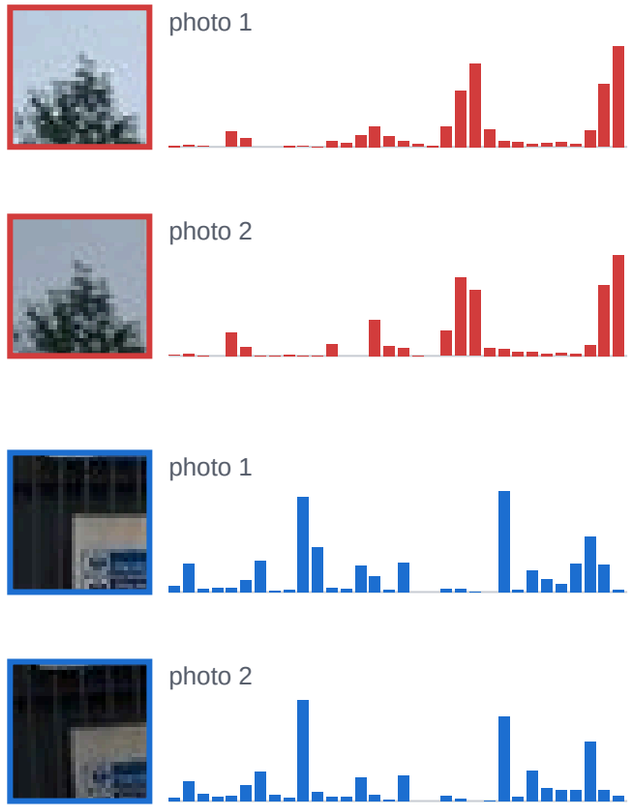
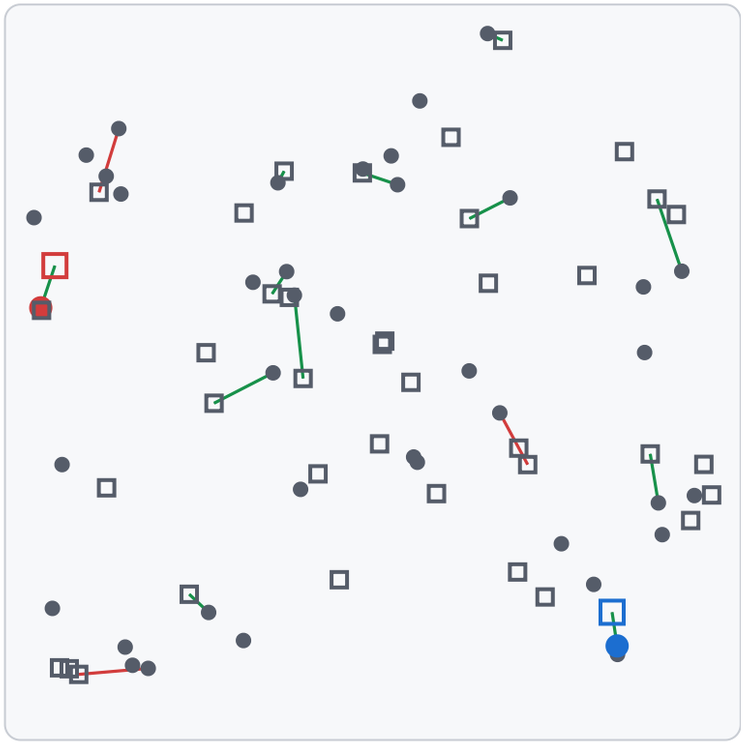


photo 2 (moved, turned, darker)





patch → vector (32 numbers: 4 cells × 8 gradient directions)



● photo 1, □ photo 2: one point per descriptor (the 32-D space, shown in 2D)

match = nearest point of the other photo:
12 right, 3 wrong (verify: RANSAC removes them)

photos: crops of “Canary Wharf”, Andreas Mischok, Poly Haven (CC0)

Finding corresponding points automatically

detect distinctive points → **describe** a vector per point → **match** nearest descriptors → **verify** RANSAC on the geometry

A good keypoint is

- **repeatable**: the same physical point is found in both images
- **well localised**: its position is precise in both directions
- **distinctive**: its neighbourhood looks unlike others

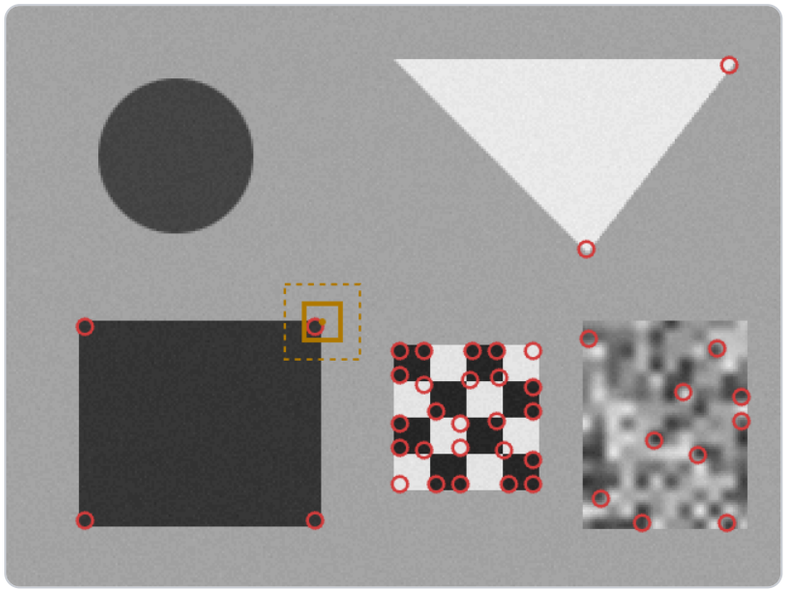
…despite

- translation, rotation, **scale** (the robot moves closer)
- brightness and contrast changes
- moderate changes of viewpoint, noise, blur

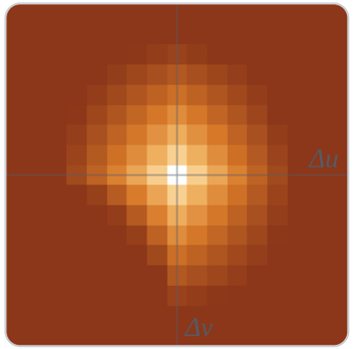
and fast enough for a robot: hundreds to thousands of points per frame, at frame rate.

What is a good keypoint?

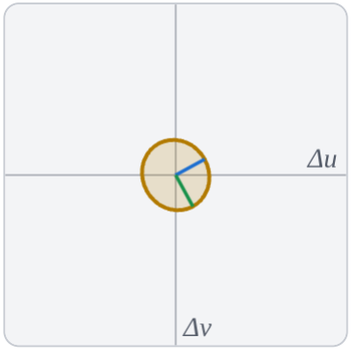
The experiment: take the small square window (orange) around a point, shift it by $(\Delta u, \Delta v)$ and measure how different it looks from the unshifted window: $E(\Delta) = \sum_W \left(I(\mathbf{x}+\Delta) - I(\mathbf{x})\right)^2$. A point is easy to find again if **every** shift makes a difference.



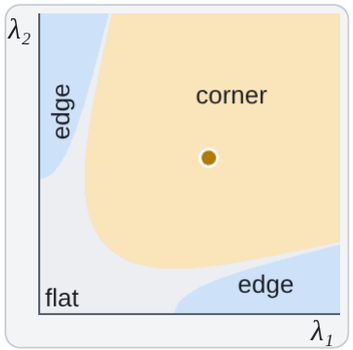
shift error E(Δu, Δv)



quadratic ≈ ΔᵀMΔ



eigenvalues of M



at (130, 130)

λ_1 = 222

λ_2 = 189

R = 33524

→ **corner**

shift error: E for all shifts within ±8 px, the centre is no shift. Dark: looks the same, bright: different. **flat**: dark everywhere (no shift is noticed). **edge**: a dark valley along the edge (sliding along it is not noticed). **corner**: dark only at the centre (every shift is noticed).

quadratic: for small shifts $E(\Delta) \approx \Delta^\top M \Delta$, a 2×2 matrix computed from the image gradients (next slide). Its ellipse summarises the error surface; the semi-axes show the eigenvalues $\lambda_1 \geq \lambda_2$.

eigenvalues of M: both small: flat; one large: edge; both large: corner. "Harris corners" marks every point of the image that passes this test.

Harris corner detector

Formally the image is a differentiable function $I : \mathbb{R}^2 \to \mathbb{R}$ (the brightness before sampling, or a smooth interpolation of the pixels); the pixels are its samples. $I_x = \partial I/\partial x$, $I_y = \partial I/\partial y$: its partial derivatives, at the pixels approximated by finite differences (e.g. Sobel). The gradient $\nabla I(\mathbf{x}) = \begin{bmatrix} I_x(\mathbf{x}) \\ I_y(\mathbf{x}) \end{bmatrix}$ is, at each pixel, a **column vector** in $\mathbb{R}^2$: so $\nabla I^\top \Delta$ is a number and $\nabla I \, \nabla I^\top$ a 2×2 matrix.

Shift the window by $\Delta \in \mathbb{R}^2$; Taylor: $I(\mathbf{x} + \Delta) \approx I(\mathbf{x}) + \nabla I(\mathbf{x})^\top \Delta$

$$E(\Delta) = \sum_{\mathbf{x} \in W} \big(I(\mathbf{x} + \Delta) - I(\mathbf{x})\big)^2 \approx \sum_{\mathbf{x} \in W} \big(\nabla I(\mathbf{x})^\top \Delta\big)^2$$
$$= \sum_{\mathbf{x} \in W} \Delta^\top \nabla I(\mathbf{x}) \, \nabla I(\mathbf{x})^\top \, \Delta = \Delta^\top M \, \Delta$$

$$M = \sum_{\mathbf{x} \in W} \nabla I \, \nabla I^\top = \sum_{\mathbf{x} \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

The structure tensor: a quadratic form again; its eigenvalues $\lambda_1 \geq \lambda_2$ say how fast E grows in the best and worst direction.

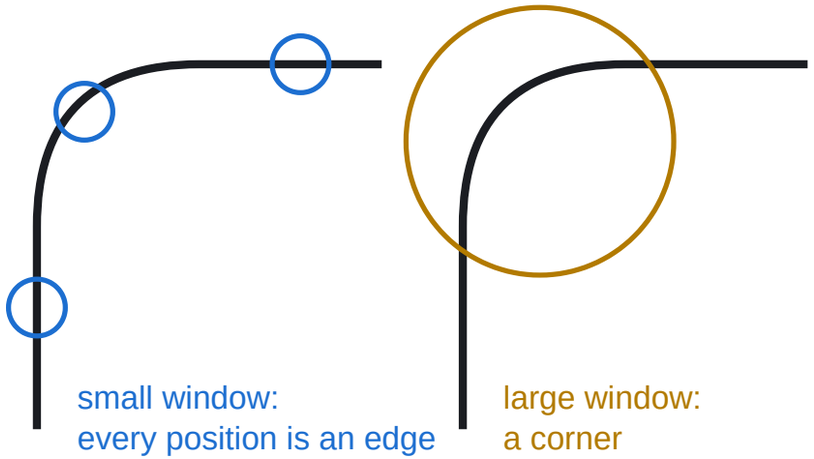
	λ_1	λ_2
flat	small	small
edge	large	small
corner	large	large

Harris & Stephens (1988), no eigenvalues needed:

$$R = \det M - k \, (\operatorname{tr} M)^2 = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2$$

$k \approx 0.04$ to 0.06; keypoints = local maxima of R above a threshold. Shi-Tomasi: $\min(\lambda_1, \lambda_2)$, `cv2.goodFeaturesToTrack`.

Scale

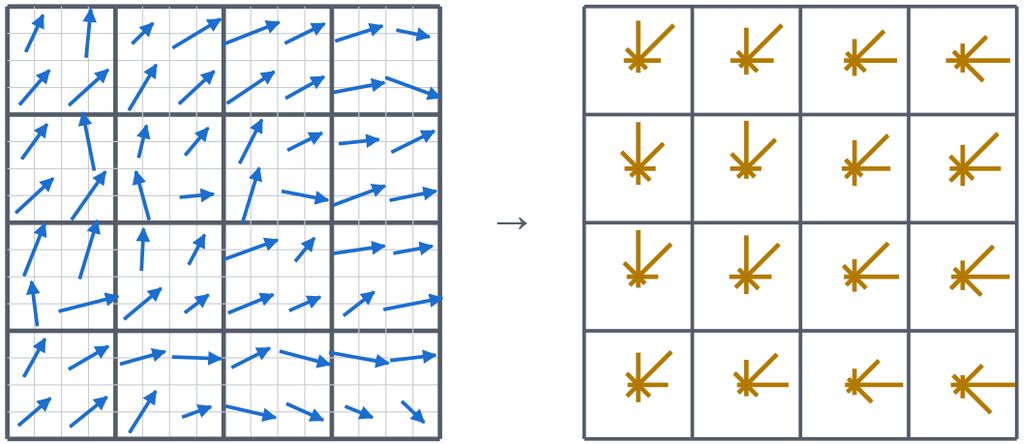


Whether a structure is a corner depends on the window size relative to the image of it: which changes when the robot moves closer.

Scale space: blur the image at many scales σ (Gaussian pyramid); look for extrema in (x, y, σ) of a blob response, e.g. the difference of Gaussians (SIFT). Each keypoint gets a position **and** a characteristic scale.

Orientation: the dominant gradient direction around the point. Describing the patch in this scaled, rotated frame makes the descriptor invariant to scale and rotation.

Descriptors: SIFT OPTIONAL



gradients in a 16×16 patch

4×4 cells × 8 orientations

→ a 128-dimensional vector, normalised to unit length

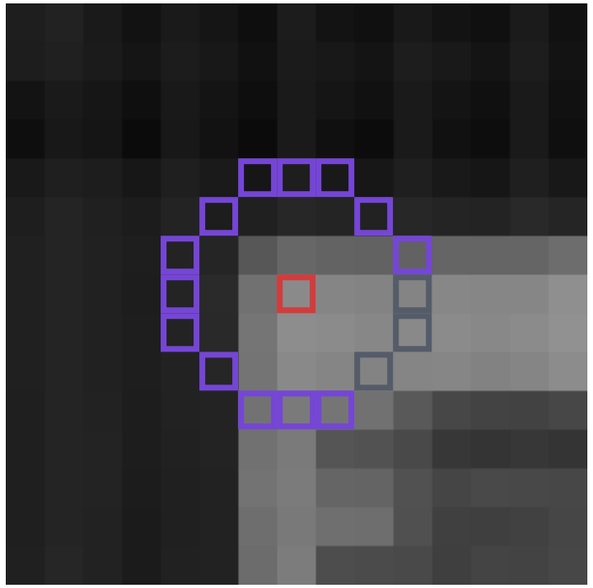
SIFT (Lowe, 1999/2004) turns the neighbourhood of a keypoint into 128 numbers, in four steps:

1. **the patch**: 16×16 samples around the keypoint, sized by its scale and turned by its orientation (previous slide), so both images give the **same physical patch**
2. **gradients** (blue arrows): at each sample, the direction and strength of the brightness change; a brightness offset does not change them
3. **histograms**: split the patch into 4×4 cells; in each cell, every arrow adds its length to one of 8 direction bins (orange stars: the length of each ray is the bin). An edge that moves by a pixel stays in its cell: robust to small shifts and deformations
4. **the vector**: 16 cells × 8 bins = 128 numbers, normalised to unit length: robust to contrast (aI multiplies all gradients by a)

Two descriptors are compared by their Euclidean distance. "Detect, describe, match" used the same idea with 2×2 cells (32 numbers).

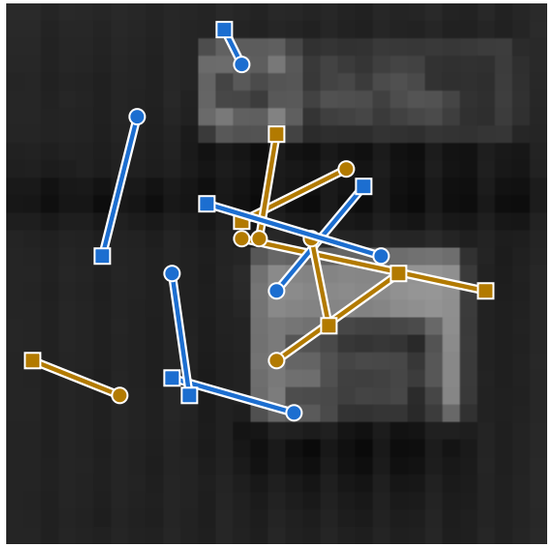
Descriptors: ORB OPTIONAL

1. FAST: is it a corner?



16 pixels on a circle around the centre (red):
brighter / darker than it by a threshold, or similar (grey).
≥ 9 in a row brighter (or darker) → a corner. Here: 13 in a row.

2. BRIEF: one bit per pair of pixels



pair i: is pixel ● a_i darker than ■ b_i?
the first 12 of 256 fixed pairs: 011101001100
256 pairs → 256 bits = 32 bytes

3. compare: count the different bits

photo 1, the point on the left:
`01110100110010010010111000101010…`

photo 2, the same point:
`01110100110010010010111011101010…`

photo 1, another point:
`10101111000011111011000110000101…`

first 32 of 256 bits; red: differs from the top row

Hamming distance (all 256 bits):
same point **21**, another point **137**
computed as popcount(a XOR b): a few instructions

ORB (Rublee et al., 2011) = oriented FAST + rotated BRIEF: FAST on an image pyramid (scale), the best corners by the Harris score; an orientation from the patch's brightness centroid, and the pairs are turned by it (photo 2 is turned by 4°). Only comparisons, 32 bytes per point, distance by XOR and popcount: one to two orders of magnitude faster than SIFT. The feature of ORB-SLAM.

photo: crop of “Canary Wharf”, Andreas Mischok, Poly Haven (CC0)

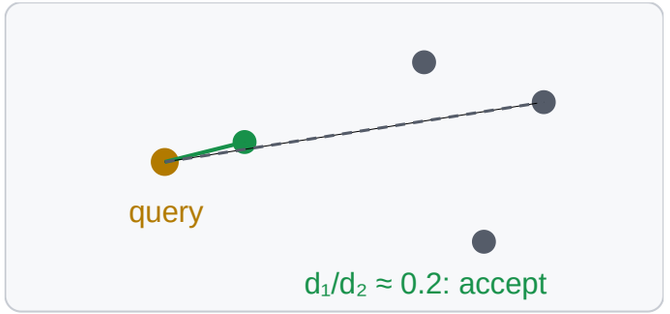
Matching

- for each descriptor in image 1: the **nearest** descriptor in image 2
- repeated structures (windows, tiles) have several near-identical neighbours
  → **ratio test** (Lowe): accept only if

$$\frac{d_1}{d_2} < 0.8$$

  d_1, d_2: distances to the nearest and the second nearest
- **mutual** check: also the nearest in the other direction
- then RANSAC on the geometry (a homography, or F in Part 3)

`cv2.BFMatcher().knnMatch(d1, d2, k=2)`; many points: approximate nearest neighbours (FLANN, FAISS).





Features today

	examples	idea
hand-designed	Harris (1988), SIFT (2004), SURF (2006), ORB (2011), AKAZE (2013)	gradients and intensity tests; fast, well understood, still the default in real-time SLAM
learned detector + descriptor	SuperPoint (2018), D2-Net, R2D2 (2019), DISK (2020), ALIKED (2023), XFeat (2024)	a CNN predicts keypoints and descriptors; trained on homographic warps or on images with known geometry
learned matcher	SuperGlue (2020), LightGlue (2023)	attention over both images' keypoints together: matches by context, not only by the nearest descriptor
detector-free (dense)	LoFTR (2021), RoMa (2024)	match everywhere, also in weak texture
3D foundation models	DUSt3R, MASt3R (2024), VGGT (2025)	a transformer predicts the 3D points (and cameras) of image pairs or sets directly; matches come as a by-product

In practice: ORB or SIFT when speed matters and the views are similar (odometry); SuperPoint + LightGlue for large changes of viewpoint, illumination or season (e.g. COLMAP / hloc pipelines).

Learned features fail differently: out of their training distribution, or when the GPU budget is the limit. RANSAC stays in every pipeline.

Part 3

Two-view geometry

a general pair of cameras, a general scene

The problem

Given: two images of a 3D scene from a camera that moved (rotated **and** translated); matched features; usually K.

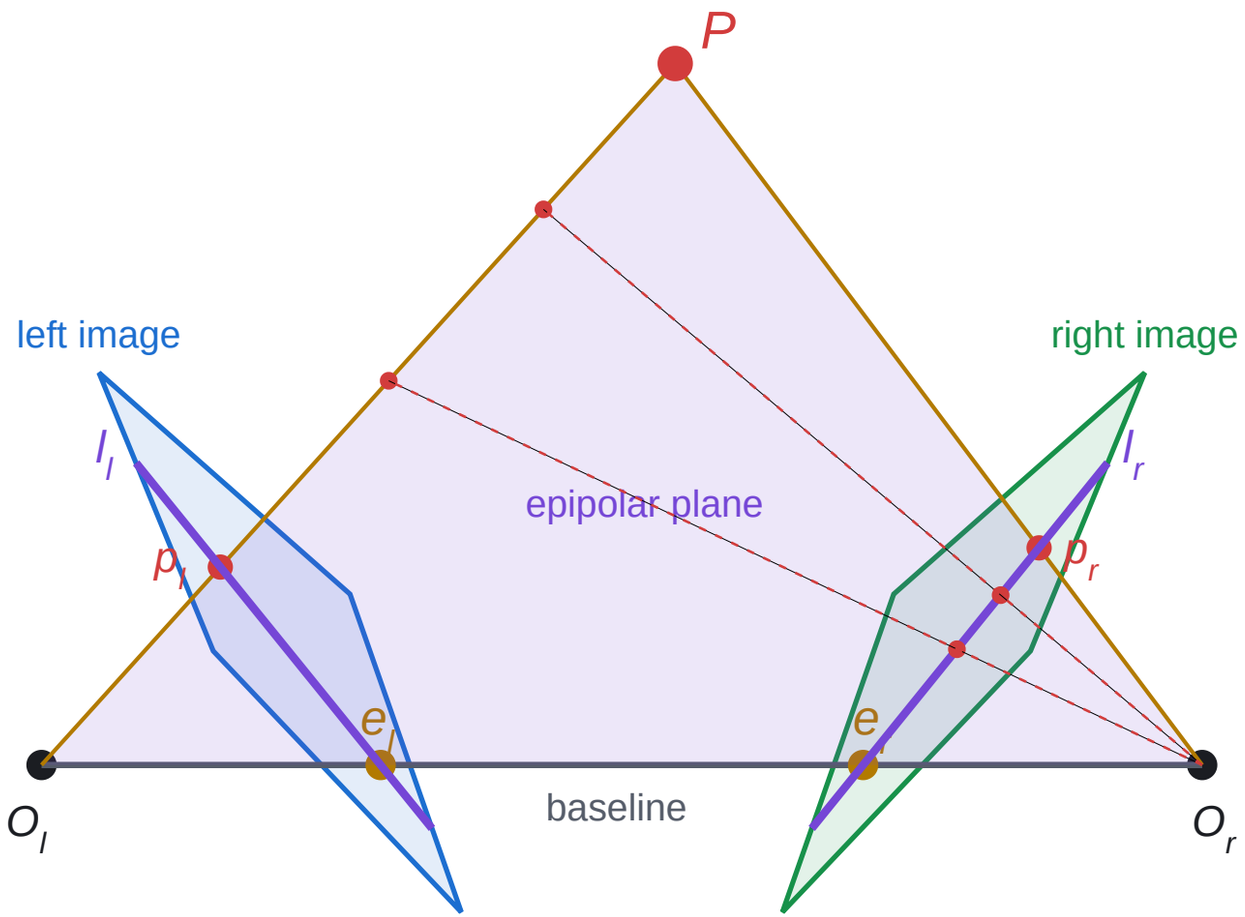
Find: the relative pose $R, \mathbf{t}$ between the two views (the camera as a motion sensor), and then the 3D points.

> **Question** No homography relates the images (parallax). Do the matches constrain anything?

Compared with lecture 2

- **PnP**: known 3D points → pose. Here: no 3D points.
- **rectified stereo**: known pose, the match lies on the **same row**. Here: unknown pose, and the match lies on a **line**: the epipolar line.

Epipolar geometry: the terms



Two cameras: centres O_l, O_r, and their image planes.

The **baseline** joins the centres. Where it pierces the image planes: the **epipoles** $\mathbf{e}_l$, $\mathbf{e}_r$. Equivalently, $\mathbf{e}_r$ is the **image of the left camera's centre** in the right camera, and vice versa.

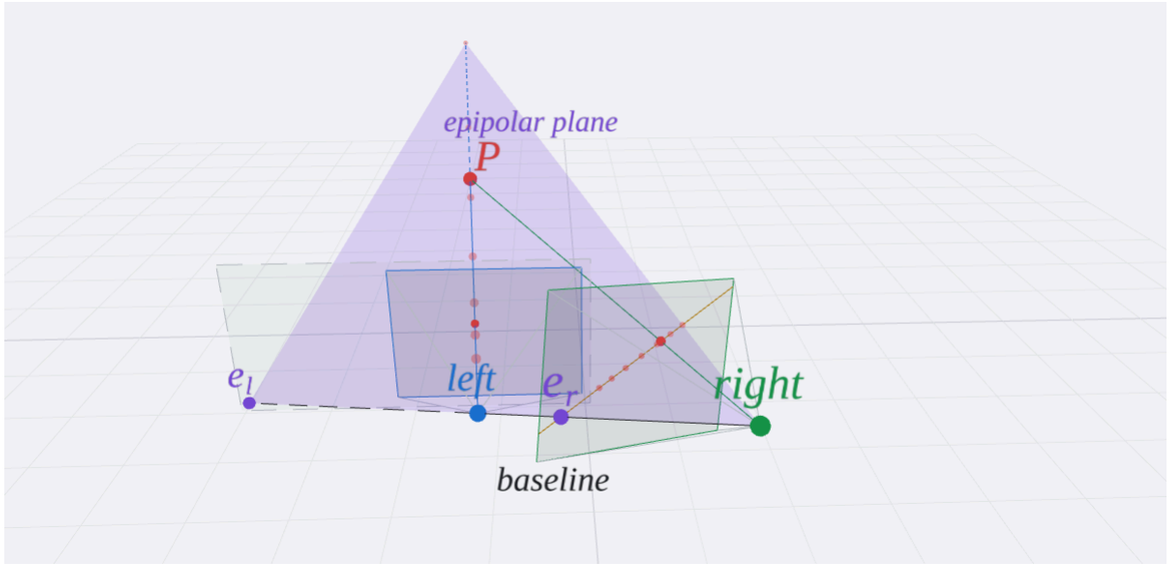
A scene point P and the two centres span a plane: the **epipolar plane** of P. Both rays, and both pixels $\mathbf{p}_l$, $\mathbf{p}_r$, lie in it.

The plane cuts the images in the **epipolar lines** $\mathbf{l}_l$, $\mathbf{l}_r$. Wherever P is on the ray of $\mathbf{p}_l$ (unknown depth), its image lies on $\mathbf{l}_r$: the match of $\mathbf{p}_l$ is on **one line**.

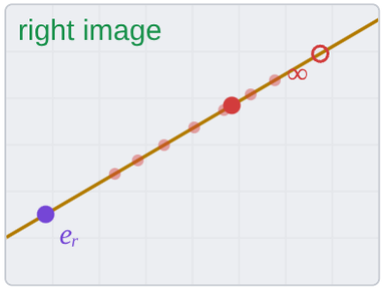
Every epipolar plane contains the baseline (the planes turn about it like the pages of a book), so **every epipolar line passes through the epipole**.

Epipolar geometry in 3D

Why it matters: matching becomes a 1D search along the line, and wrong matches can be rejected; the line depends only on the relative pose $R, \mathbf{t}$ (and K), so every correct match is a **measurement of the camera motion** (Part 4).







$\tilde{\mathbf{u}}_l = (280,\ 200,\ 1)$

epipolar line $\mathbf{l}_r = F\tilde{\mathbf{u}}_l$:

$0.505\,u + 0.863\,v - 344.5 = 0$

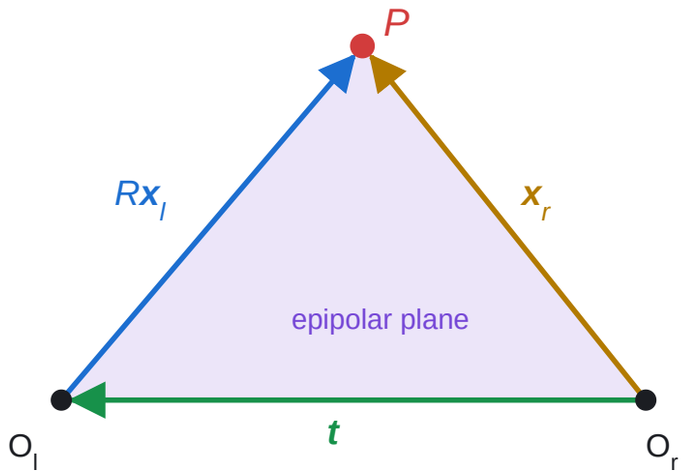
P in the right image:

$\tilde{\mathbf{u}}_r = (387.5,\ 172.4,\ 1)$

$\tilde{\mathbf{u}}_r^\top F\,\tilde{\mathbf{u}}_l = 0$ for every depth

epipole $e_r = (68,\ 359)$

The epipolar constraint



Point in the left camera frame $\mathbf{x}_l$, in the right frame $\mathbf{x}_r = R\,\mathbf{x}_l + \mathbf{t}$ (all vectors in the **right** camera's axes):

- $\mathbf{t}$: from O_r to O_l (put $\mathbf{x}_l = \mathbf{0}$)
- $\mathbf{x}_r$: from O_r to P; $R\mathbf{x}_l = \mathbf{x}_r - \mathbf{t}$: from O_l to P

All three lie in the epipolar plane. $\mathbf{t} \times R\mathbf{x}_l$ is perpendicular to both $\mathbf{t}$ and $R\mathbf{x}_l$, so it is the **normal of the epipolar plane**, and $\mathbf{x}_r$, lying in the plane, is perpendicular to it:

$$\mathbf{x}_r^\top (\mathbf{t} \times R\,\mathbf{x}_l) = 0 \quad \Longrightarrow \quad \mathbf{x}_r^\top \underbrace{[\mathbf{t}]_\times R}_{E}\, \mathbf{x}_l = 0$$

$$[\mathbf{t}]_\times = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}, \qquad [\mathbf{t}]_\times \mathbf{a} = \mathbf{t} \times \mathbf{a}$$

The essential matrix $E = [\mathbf{t}]_\times R$

$$\mathbf{x}_r^\top E\, \mathbf{x}_l = 0$$

- **no depth needed**: we know only the pixel, not the 3D point $\mathbf{x}_l$. But replacing $\mathbf{x}_l$ by $\lambda\mathbf{x}_l$ multiplies the left side by λ, and 0 stays 0. So any point on the ray will do, e.g. the one at depth 1: $\hat{\mathbf{x}}_l = K^{-1}\tilde{\mathbf{u}}_l = (x_l/z_l,\ y_l/z_l,\ 1)$; the same for $\mathbf{x}_r$: $\hat{\mathbf{x}}_r^\top E\, \hat{\mathbf{x}}_l = 0$
- $E\mathbf{x}_l$ is the epipolar line in the right (normalised) image: one equation, one line
- E is defined up to scale; $\det E = 0$ (rank 2)
- **5 DoF**: 3 (rotation) + 3 (translation) − 1 (scale)

Example: rectified stereo

Right camera at $(b, 0, 0)$, same orientation: $R = I$, $\mathbf{t} = (-b, 0, 0)$.

$$E = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & b \\ 0 & -b & 0 \end{bmatrix}, \quad \mathbf{x}_r^\top E\, \mathbf{x}_l = b\,(y_r z_l - z_r y_l) = 0$$

$\Rightarrow\ y_r/z_r = y_l/z_l$: the **same row**, as in lecture 2.

Pixels: the fundamental matrix F

Pixels: $\tilde{\mathbf{u}}_l \equiv K_l \mathbf{x}_l$, $\tilde{\mathbf{u}}_r \equiv K_r \mathbf{x}_r$. Substitute $\mathbf{x}_l \equiv K_l^{-1} \tilde{\mathbf{u}}_l$, $\mathbf{x}_r \equiv K_r^{-1} \tilde{\mathbf{u}}_r$:

$$\tilde{\mathbf{u}}_r^\top \underbrace{K_r^{-\top} E \, K_l^{-1}}_{F} \tilde{\mathbf{u}}_l = 0$$

- F needs **no calibration**: estimated from pixel matches alone
- **7 DoF**: up to scale, $\det F = 0$
- epipolar line in the right image: $\mathbf{l}_r = F \tilde{\mathbf{u}}_l$, i.e. $l_1 u + l_2 v + l_3 = 0$
- epipoles: $F \, \tilde{\mathbf{e}}_l = \mathbf{0}$, $F^\top \tilde{\mathbf{e}}_r = \mathbf{0}$ (every epipolar line passes through them)

Known K (a robot's camera)

$E = K_r^\top F K_l$: estimate E directly from normalised points. Fewer DoF (5), more stable, and E contains the pose.

Unknown K (photos from the internet)

F only; the reconstruction is then determined up to a projective transformation (calibration adds the metric structure).

Estimating F: the eight-point algorithm

Each match $(u_l, v_l) \leftrightarrow (u_r, v_r)$ gives one equation, **linear** in the entries of F:

$$\begin{bmatrix} u_r u_l & u_r v_l & u_r & v_r u_l & v_r v_l & v_r & u_l & v_l & 1 \end{bmatrix} \mathbf{f} = 0$$

1. stack $n \geq 8$ matches: $A\mathbf{f} = \mathbf{0}$, $\min \|A\mathbf{f}\|$ s.t. $\|\mathbf{f}\| = 1$ (third time today)
2. **enforce rank 2**: SVD $F = U\Sigma V^\top$, set $\sigma_3 = 0$
3. normalise the pixel coordinates first (Hartley, "In defense of the eight-point algorithm", 1997)

With outliers and known K

- RANSAC with the **minimal** solver: 7 points for F, **5 points** for E (Nistér, 2004)
- inlier test: distance of $\tilde{\mathbf{u}}_r$ from the line $F\tilde{\mathbf{u}}_l$ (Sampson distance)
- `cv2.findFundamentalMat(p1, p2, cv2.FM_RANSAC)`
  `cv2.findEssentialMat(p1, p2, K, cv2.RANSAC)`

With 50% outliers ($p = 0.99$): $s = 8$ needs 1177 iterations, $s = 5$ only 146.

From E to the pose OPTIONAL

SVD: $E = U\ \mathrm{diag}(1,1,0)\,V^\top$ (with $\det U = \det V = +1$), and

$$W = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad R = UWV^\top \ \text{ or } \ UW^\top V^\top, \quad \mathbf{t} = \pm\,\mathbf{u}_3$$

$\mathbf{u}_3$: the third column of U (the null vector of $E^\top$).

- so E gives $\mathbf{t}$ (up to sign and scale) and two possible R: **four** candidate poses
- only one puts the triangulated points **in front of both cameras** (positive depth, "cheirality")
- as with P3P's four solutions (lecture 2): an extra check picks the right one

`cv2.recoverPose(E, p1, p2, K)` → R, $\mathbf{t}$ with $\|\mathbf{t}\| = 1$

.

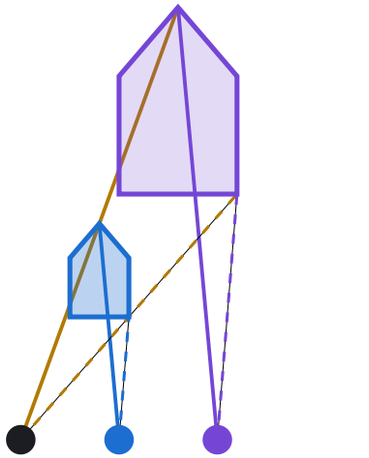
Triangulation

Known $P_l = K[I \mid \mathbf{0}]$, $P_r = K[R \mid \mathbf{t}]$ and a match: intersect the two rays. With noise they miss each other:

- DLT: $\tilde{\mathbf{u}} \times P\tilde{\mathbf{X}} = \mathbf{0}$, two equations per view, $A\tilde{\mathbf{X}} = \mathbf{0}$ (fourth time)
- better: minimise the reprojection error
- poor when the rays are almost parallel (small baseline, far point): lecture 2, $\Delta z \sim z^2$

`cv2.triangulatePoints(P1, P2, p1, p2)`

The missing scale



Scale the scene **and** the camera motion by the same factor k: every ray keeps its direction, the images are identical.

- E (and the pose from it) is known only up to scale: $\|\mathbf{t}\| = 1$ by convention
- a single moving camera cannot tell a toy house 1 m away from a real one 20 m away

Where the scale comes from

- a stereo pair: the known baseline (lecture 2)
- an IMU (visual-inertial odometry: accelerations are metric)
- wheel odometry, a known object size, a depth sensor

Film makers rely on it

Miniatures: spaceships, cities and disasters were filmed as models with a small camera moving slowly (motion control), and the audience could not tell. The "bigatures" of The Lord of the Rings: Minas Tirith at 1/72 scale. What gives a miniature away is not geometry but physics and optics (water, fire, falling objects, a shallow depth of field), so miniatures are filmed at a higher frame rate, about $\sqrt{1/\text{scale}}$ times normal, and played back slowed down.

Video: Behind the biggest movies' tiniest sets (ScreenRant)

Part 4

Visual odometry

and why it is not enough

Visual odometry: the pipeline

1. Initialisation (once, at the start: no 3D points exist yet, so only 2D-2D matches and E)

first frames → features: detect + match / track → first two frames: E + RANSAC → $R, \mathbf{t}$ ($\|\mathbf{t}\| = 1$) → triangulate 3D points

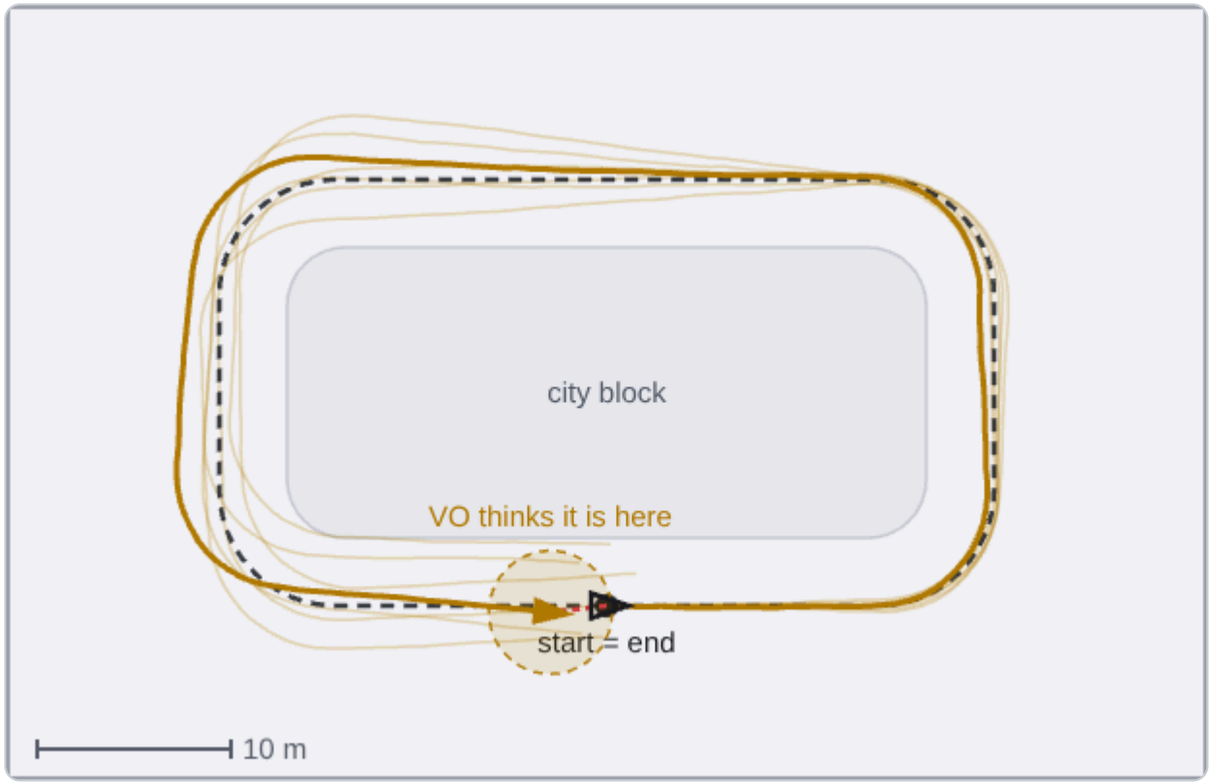
2. Tracking (every new frame: the 3D points now exist, so 2D-3D matches and PnP)

new frame → **PnP** + RANSAC on the known 3D points (lecture 2) → triangulate new points → local refinement (bundle adjustment over the last frames) → camera trajectory

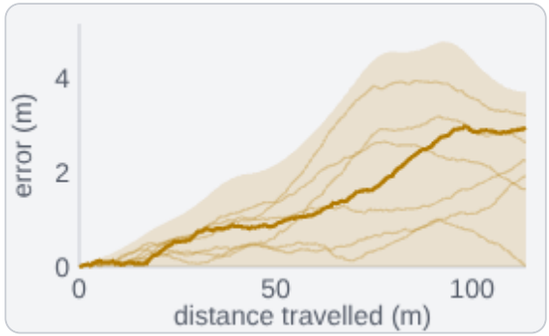
- the camera as a **motion sensor**: frame-to-frame pose, chained over time
- why not E for every pair: each pair would get its own unknown scale ($\|\mathbf{t}\| = 1$ every time); PnP against the triangulated points keeps **one** scale, and uses more information
- stereo VO: triangulate from the stereo pair each frame: metric, no scale problem

Nistér et al. coined "visual odometry" (2004). The Mars rovers Spirit and Opportunity used stereo VO to correct wheel slip (Maimone et al., 2007).

Why odometry is not enough: drift



step **160** / 160, driven **114 m**
VO error: **2.9 m**, heading 5.1°
back at the start, but VO does not know it



- true path, △ true pose
- visual odometry, ▲
  faint: other noise draws
- 95% uncertainty region
- after loop closure

From odometry to SLAM

SLAM: simultaneous localisation and mapping. Keep a map, and use it:

- **loop closure**: recognise a place seen before (bag of visual words, learned place recognition), add a constraint "this pose ≈ that pose"
- **optimise** the whole trajectory (pose graph) or the poses and the points together (bundle adjustment)
- **relocalise** after losing track (PnP against the map)

Systems

- ORB-SLAM3 (2021): features, mono / stereo / RGB-D, with an IMU
- VINS-Mono (2018): visual-inertial
- DROID-SLAM (2021): learned, dense
- MASt3R-SLAM (2025): on top of a 3D foundation model

Next lecture (Oct 28): 3D reconstruction and SLAM in depth.

Summary

- a homography relates two images of a **plane**, or two images taken by a **rotating** camera: $H = KRK^{-1}$; otherwise parallax
- H: 8 DoF, 4 points, DLT + RANSAC; stitching = warp into a reference frame + blend
- features: repeatable, well localised, distinctive; Harris (M, two large eigenvalues), SIFT, ORB; ratio test; learned features and matchers
- general pair: the match lies on the **epipolar line**; $\mathbf{x}_r^\top E\,\mathbf{x}_l = 0$, $E = [\mathbf{t}]_\times R$ (5 DoF), $F = K_r^{-\top} E K_l^{-1}$ (7 DoF)
- 8-point / 5-point + RANSAC → E → four poses, cheirality picks one; triangulation
- monocular: **no scale**; visual odometry drifts → loop closure, SLAM

> One recipe all along: write the model as linear equations, solve $\min \|A\mathbf{x}\|$ with $\|\mathbf{x}\| = 1$, wrap it in RANSAC, refine the error in pixels.

Exercises

1. Write the two DLT rows for the correspondence $(1, 2) \to (3, 5)$. Why must no three of the four points be collinear?
2. K: $f = 500$ px, principal point $(320, 240)$. The camera pans (turns about its vertical axis) by 10°. Roughly how far does the image centre move? Show that $\tilde{\mathbf{u}}_2 \equiv K R K^{-1} \tilde{\mathbf{u}}_1$.
3. Rectified stereo, right camera at $(b, 0, 0)$: compute E and show that the epipolar constraint gives $v_l = v_r$.
4. Why is $\det E = 0$? Which point does $[\mathbf{t}]_\times$ send to zero, and what is its image in the right camera?
5. RANSAC with 50% outliers, $p = 0.99$: how many iterations for the 8-point and the 5-point solver?
6. A monocular VO system reports that the robot drove 3.0 units. Name two sensors that would turn this into metres.

Feedback



Please spend a minute: your feedback helps me improve the next lecture.

`tinyurl.com/mimuw-rc-feedback`

Anonymous, no login, three questions.